

## Chapter 4

# Similarity-based Learning

*"Anyone who stops learning is old, whether at twenty or eighty."*  
— Henry Ford

*Similarity-based Learning* is a supervised learning technique that predicts the class label of a test instance by gauging the similarity of this test instance with training instances. Similarity-based learning refers to a family of instance-based learning which is used to solve both classification and regression problems. Instance-based learning makes prediction by computing distances or similarities between test instance and specific set of training instances local to the test instance in an incremental process. In contrast to other learning mechanisms, it considers only the nearest instance or instances to predict the class of unseen instances. This learning methodology improves the performance of classification since it uses only a specific set of instances as incremental learning task. Similarity-based classification is useful in various fields such as image processing, text classification, pattern recognition, bio informatics, data mining, information retrieval, natural language processing, etc. A practical application of this learning is predicting daily stock index price changes. This chapter provides an insight of how different similarity-based models predict the class of a new instance.

### Learning Objectives

- Understand the fundamentals of Instance based learning
- Know about the concepts of Nearest-Neighbor Learning using the algorithm called  $k$ -Nearest-Neighbors ( $k$ -NN)
- Learn about Weighted  $k$ -Nearest-Neighbor classifier that chooses the neighbors by using the weighted distance
- Gain knowledge about Nearest Centroid classifier, a simple alternative to  $k$ -NN classifiers
- Understand Locally Weighted Regression (LWR) that approximates the linear functions of all  $k$  neighbors to minimize the error while prediction

## 4.2 NEAREST-NEIGHBOR LEARNING

A natural approach to similarity-based classification is  $k$ -Nearest-Neighbors ( $k$ -NN), which is a non-parametric method used for both classification and regression problems. It is a simple and powerful non-parametric algorithm that predicts the category of the test instance according to the ' $k$ ' training samples which are closer to the test instance and classifies it to that category which has the largest probability. A visual representation of this learning is shown in Figure 4.1. There are two classes of objects called  $C_1$  and  $C_2$  in the given figure. When given a test instance  $T$ , the category of this test instance is determined by looking at the class of  $k = 3$  nearest neighbors. Thus, the class of this test instance  $T$  is predicted as  $C_2$ .

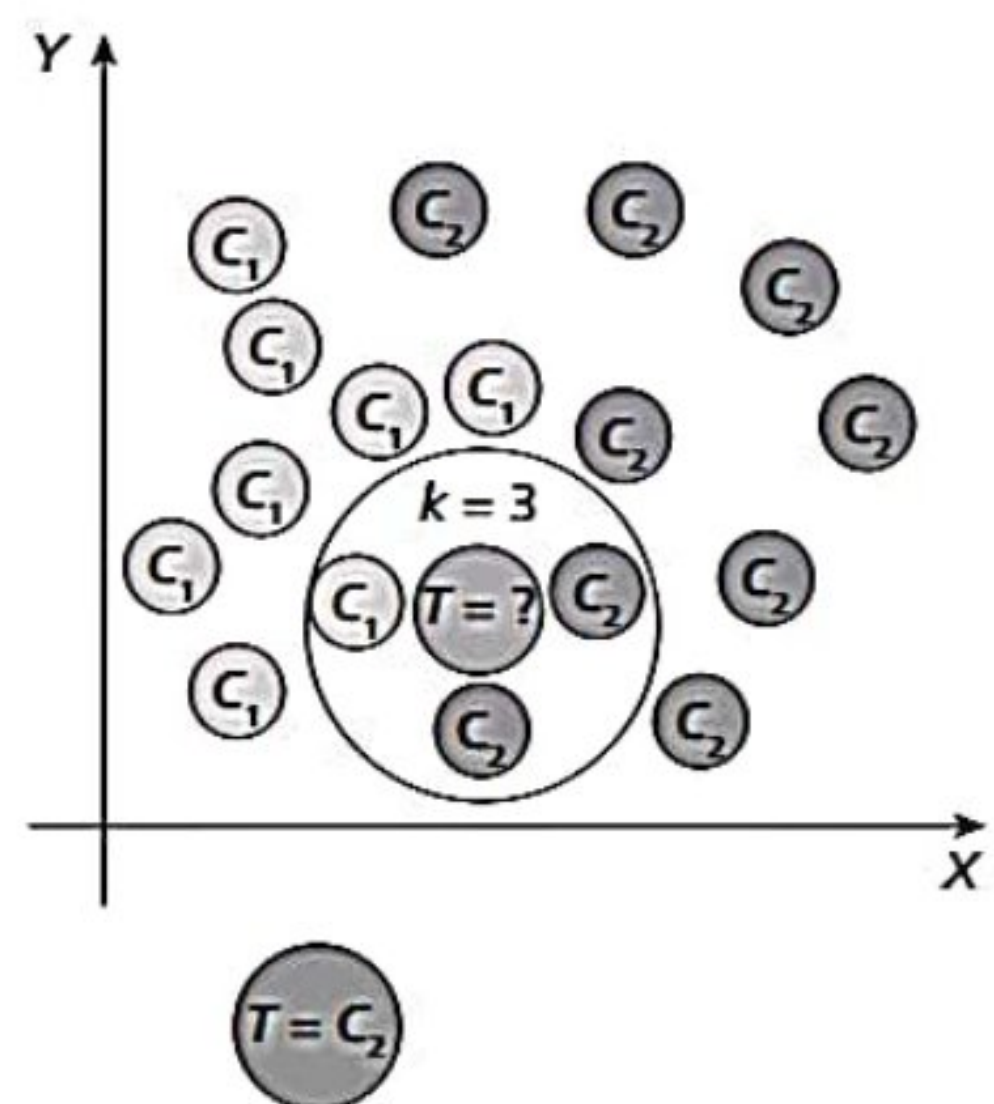


Figure 4.1: Visual Representation of  $k$ -Nearest Neighbor Learning

The algorithm relies on the assumption that similar objects are close to each other in the feature space.  $k$ -NN performs instance-based learning which just stores the training data instances and learning instances case by case. The model is also 'memory-based' as it uses training data at time when predictions need to be made. It is a lazy learning algorithm since no prediction model is built earlier with training instances and classification happens only after getting the test instance.

The algorithm classifies a new instance by determining the ' $k$ ' most similar instances (i.e.,  $k$  nearest neighbors) and summarizing the output of those ' $k$ ' instances. If the target variable is discrete then it is a classification problem, so it selects the most common class value among the ' $k$ ' instances by a majority vote. However, if the target variable is continuous then it is a regression problem, and hence the mean output variable of the ' $k$ ' instances is the output of the test instance.

The most popular distance measure such as Euclidean distance is used in  $k$ -NN to determine the ' $k$ ' instances which are similar to the test instance. The value of ' $k$ ' is best determined by tuning with different ' $k$ ' values and choosing the ' $k$ ' which classifies the test instance more accurately.

#### Algorithm 4.1: $k$ -NN

**Inputs:** Training dataset  $T$ , distance metric  $d$ , Test instance  $t$ , the number of nearest neighbors  $k$

**Output:** Predicted class or category

**Prediction:** For test instance  $t$ ,

1. For each instance  $i$  in  $T$ , compute the distance between the test instance  $t$  and every other instance  $i$  in the training dataset using a distance metric (Euclidean distance).  
[Continuous attributes - Euclidean distance between two points in the plane with coordinates  $(x_1, y_1)$  and  $(x_2, y_2)$  is given as  $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ ]  
[Categorical attributes (Binary) - Hamming Distance: If the value of the two instances is same, the distance  $d$  will be equal to 0 otherwise  $d = 1$ .]
2. Sort the distances in an ascending order and select the first  $k$  nearest training data instances to the test instance.
3. Predict the class of the test instance by majority voting (if target attribute is discrete valued) or mean (if target attribute is continuous valued) of the  $k$  selected nearest instances.

**Example 4.1:** Consider the student performance training dataset of 8 data instances shown in Table 4.2 which describes the performance of individual students in a course and their CGPA obtained in the previous semesters. The independent attributes are CGPA, Assessment and Project. The target variable is 'Result' which is a discrete valued variable that takes two values 'Pass' or 'Fail'. Based on the performance of a student, classify whether a student will pass or fail in that course.

Table 4.2: Training Dataset  $T$

| S.No. | CGPA | Assessment | Project Submitted | Result |
|-------|------|------------|-------------------|--------|
| 1.    | 9.2  | 85         | 8                 | Pass   |
| 2.    | 8    | 80         | 7                 | Pass   |
| 3.    | 8.5  | 81         | 8                 | Pass   |

(Continued)

| S.No. | CGPA | Assessment | Project Submitted | Result |
|-------|------|------------|-------------------|--------|
| 4.    | 6    | 45         | 5                 | Fail   |
| 5.    | 6.5  | 50         | 4                 | Fail   |
| 6.    | 8.2  | 72         | 7                 | Pass   |
| 7.    | 5.8  | 38         | 5                 | Fail   |
| 8.    | 8.9  | 91         | 9                 | Pass   |

**Solution:** Given a test instance (6.1, 40, 5) and a set of categories {Pass, Fail} also called as classes, we need to use the training set to classify the test instance using Euclidean distance.

The task of classification is to assign a category or class to an arbitrary instance.

Assign  $k = 3$ .

**Step 1:** Calculate the Euclidean distance between the test instance (6.1, 40, and 5) and each of the training instances as shown in Table 4.3.

**Table 4.3:** Euclidean Distance

| S.No. | CGPA | Assessment | Project Submitted | Result | Euclidean Distance                                             |
|-------|------|------------|-------------------|--------|----------------------------------------------------------------|
| 1.    | 9.2  | 85         | 8                 | Pass   | $\sqrt{(9.2 - 6.1)^2 + (85 - 40)^2 + (8 - 5)^2}$<br>= 45.2063  |
| 2.    | 8    | 80         | 7                 | Pass   | $\sqrt{(8 - 6.1)^2 + (80 - 40)^2 + (7 - 5)^2}$<br>= 40.09501   |
| 3.    | 8.5  | 81         | 8                 | Pass   | $\sqrt{(8.5 - 6.1)^2 + (81 - 40)^2 + (8 - 5)^2}$<br>= 41.17961 |
| 4.    | 6    | 45         | 5                 | Fail   | $\sqrt{(6 - 6.1)^2 + (45 - 40)^2 + (5 - 5)^2}$<br>= 5.001      |
| 5.    | 6.5  | 50         | 4                 | Fail   | $\sqrt{(6.5 - 6.1)^2 + (50 - 40)^2 + (4 - 5)^2}$<br>= 10.05783 |
| 6.    | 8.2  | 72         | 7                 | Pass   | $\sqrt{(8.2 - 6.1)^2 + (72 - 40)^2 + (7 - 5)^2}$<br>= 32.13114 |
| 7.    | 5.8  | 38         | 5                 | Fail   | $\sqrt{(5.8 - 6.1)^2 + (38 - 40)^2 + (5 - 5)^2}$<br>= 2.022375 |
| 8.    | 8.9  | 91         | 9                 | Pass   | $\sqrt{(8.9 - 6.1)^2 + (91 - 40)^2 + (9 - 5)^2}$<br>= 51.23319 |

**Step 2:** Sort the distances in the ascending order and select the first 3 nearest training data instances to the test instance. The selected nearest neighbors are shown in Table 4.4.

**Table 4.4:** Nearest Neighbors

| Instance | Euclidean Distance | Class |
|----------|--------------------|-------|
| 4        | 5.001              | Fail  |
| 5        | 10.05783           | Fail  |
| 7        | 2.022375           | Fail  |

Here, we take the 3 nearest neighbors as instances 4, 5 and 7 with smallest distances.

**Step 3:** Predict the class of the test instance by majority voting.

The class for the test instance is predicted as 'Fail'.

Data normalization/standardization is required when data (features) have different ranges or a wider range of possible values when computing distances and to transform all features to a specific range. This is probably done to eliminate the influence of one feature over another (i.e., to give all features equal chances). For example if one feature has values in the range of [0–1] and another feature has values in the range of [0–100], then the second feature will influence more even if there is a small variation than the first feature.

$k$ -NN classifier performance is strictly affected by three factors such as the number of nearest neighbors (i.e., selection of  $k$ ), distance metric and decision rule.

If the  $k$  value selected is small then it may result in overfitting or less stable and if it is big then it may include many irrelevant points from other classes. The choice of the distance metric selected also plays a major role and it depends on the type of the independent attributes in the training dataset.

The  $k$ -NN classification algorithm best suits lower dimensional data as in a high-dimensional space the nearest neighbors may not be very close at all.

### 4.3 WEIGHTED $k$ -NEAREST-NEIGHBOR ALGORITHM

The Weighted  $k$ -NN is an extension of  $k$ -NN. It chooses the neighbors by using the weighted distance. The  $k$ -Nearest Neighbor ( $k$ -NN) algorithm has some serious limitations as its performance is solely dependent on choosing the  $k$  nearest neighbors, the distance metric used and the decision rule. However, the principle idea of Weighted  $k$ -NN is that  $k$  closest neighbors to the test instance are assigned a higher weight in the decision as compared to neighbors that are farther away from the test instance. The idea is that weights are inversely proportional to distances.

The selected  $k$  nearest neighbors can be assigned uniform weights, which means all the instances in each neighborhood are weighted equally or weights can be assigned by the inverse of their distance. In the second case, closer neighbors of a query point will have a greater influence than neighbors which are further away.

#### Algorithm 4.2: Weighted $k$ -NN

**Inputs:** Training dataset ' $T$ ', Distance metric ' $d$ ', Weighting function  $w(i)$ , Test instance ' $t$ ', the number of nearest neighbors ' $k$ '

(Continued)

**Output:** Predicted class or category

**Prediction:** For test instance  $t$ ,

1. For each instance ' $i$ ' in Training dataset  $T$ , compute the distance between the test instance  $t$  and every other instance ' $i$ ' using a distance metric (Euclidean distance).  
[Continuous attributes - Euclidean distance between two points in the plane with coordinates  $(x_1, y_1)$  and  $(x_2, y_2)$  is given as  $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$  ]  
[Categorical attributes (Binary) - Hamming Distance: If the values of two instances are the same, the distance  $d$  will be equal to 0. Otherwise  $d = 1$ .]
2. Sort the distances in the ascending order and select the first ' $k$ ' nearest training data instances to the test instance.
3. Predict the class of the test instance by weighted voting technique (Weighting function  $w(i)$ ) for the  $k$  selected nearest instances:
  - Compute the inverse of each distance of the ' $k$ ' selected nearest instances.
  - Find the sum of the inverses.
  - Compute the weight by dividing each inverse distance by the sum. (Each weight is a vote for its associated class).
  - Add the weights of the same class.
  - Predict the class by choosing the class with the maximum vote.

**Example 4.2:** Consider the same training dataset given in Table 4.1. Use Weighted  $k$ -NN and determine the class.

**Solution:**

**Step 1:** Given a test instance  $(7.6, 60, 8)$  and a set of classes {Pass, Fail}, use the training dataset to classify the test instance using Euclidean distance and weighting function.

Assign  $k = 3$ . The distance calculation is shown in Table 4.5.

**Table 4.5:** Euclidean Distance

| S.No. | CGPA | Assessment | Project Submitted | Result | Euclidean Distance                                               |
|-------|------|------------|-------------------|--------|------------------------------------------------------------------|
| 1.    | 9.2  | 85         | 8                 | Pass   | $\sqrt{(9.2 - 7.6)^2 + (85 - 60)^2 + (8 - 8)^2}$<br>$= 25.05115$ |
| 2.    | 8    | 80         | 7                 | Pass   | $\sqrt{(8 - 7.6)^2 + (80 - 60)^2 + (7 - 8)^2}$<br>$= 20.02898$   |
| 3.    | 8.5  | 81         | 8                 | Pass   | $\sqrt{(8.5 - 7.6)^2 + (81 - 60)^2 + (8 - 8)^2}$<br>$= 21.01928$ |

(Continued)

| S.No. | CGPA | Assessment | Project Submitted | Result | Euclidean Distance                                       |
|-------|------|------------|-------------------|--------|----------------------------------------------------------|
| 4.    | 6    | 45         | 5                 | Fail   | $\sqrt{(6-7.6)^2 + (45-60)^2 + (5-8)^2}$<br>= 15.38051   |
| 5.    | 6.5  | 50         | 4                 | Fail   | $\sqrt{(6.5-7.6)^2 + (50-60)^2 + (4-8)^2}$<br>= 10.82636 |
| 6.    | 8.2  | 72         | 7                 | Pass   | $\sqrt{(8.2-7.6)^2 + (72-60)^2 + (7-8)^2}$<br>= 12.05653 |
| 7.    | 5.8  | 38         | 5                 | Fail   | $\sqrt{(5.8-7.6)^2 + (38-60)^2 + (5-8)^2}$<br>= 22.27644 |
| 8.    | 8.9  | 91         | 9                 | Pass   | $\sqrt{(8.9-7.6)^2 + (91-60)^2 + (9-8)^2}$<br>= 31.04336 |

**Step 2:** Sort the distances in the ascending order and select the first 3 nearest training data instances to the test instance. The selected nearest neighbors are shown in Table 4.6.

**Table 4.6: Nearest Neighbors**

| Instance | Euclidean Distance | Class |
|----------|--------------------|-------|
| 4        | 15.38051           | Fail  |
| 5        | 10.82636           | Fail  |
| 6        | 12.05653           | Pass  |

**Step 3:** Predict the class of the test instance by weighted voting technique from the 3 selected nearest instances.

- Compute the inverse of each distance of the 3 selected nearest instances as shown in Table 4.7.

**Table 4.7: Inverse Distance**

| Instance | Euclidean Distance | Inverse Distance | Class |
|----------|--------------------|------------------|-------|
| 4        | 15.38051           | 0.06502          | Fail  |
| 5        | 10.82636           | 0.092370         | Fail  |
| 6        | 12.05653           | 0.08294          | Pass  |

- Find the sum of the inverses.  
Sum = 0.06502 + 0.092370 + 0.08294 = 0.24033
- Compute the weight by dividing each inverse distance by the sum as shown in Table 4.8.

Table 4.8: Weight Calculation

| Instance | Euclidean Distance | Inverse Distance | Weight = Inverse distance/Sum | Class |
|----------|--------------------|------------------|-------------------------------|-------|
| 4        | 15.38051           | 0.06502          | 0.270545                      | Fail  |
| 5        | 10.82636           | 0.092370         | 0.384347                      | Fail  |
| 6        | 12.05653           | 0.08294          | 0.345109                      | Pass  |

- Add the weights of the same class.  
 $\text{Fail} = 0.270545 + 0.384347 = 0.654892$   
 $\text{Pass} = 0.345109$
- Predict the class by choosing the class with the maximum vote.  
The class is predicted as 'Fail'.

## 4.4 NEAREST CENTROID CLASSIFIER

A simple alternative to  $k$ -NN classifiers for similarity-based classification is the Nearest Centroid Classifier. It is a simple classifier and also called as Mean Difference classifier. The idea of this classifier is to classify a test instance to the class whose centroid/mean is closest to that instance.

### Algorithm 4.3: Nearest Centroid Classifier

**Inputs:** Training dataset  $T$ , Distance metric  $d$ , Test instance  $t$

**Output:** Predicted class or category

1. Compute the mean/centroid of each class.
2. Compute the distance between the test instance and mean/centroid of each class (Euclidean Distance).
3. Predict the class by choosing the class with the smaller distance.

**Example 4.3:** Consider the sample data shown in Table 4.9 with two features  $x$  and  $y$ . The target classes are 'A' or 'B'. Predict the class using Nearest Centroid Classifier.

Table 4.9: Sample Data

| $X$ | $Y$ | Class |
|-----|-----|-------|
| 3   | 1   | A     |
| 5   | 2   | A     |
| 4   | 3   | A     |
| 7   | 6   | B     |
| 6   | 7   | B     |
| 8   | 5   | B     |

**Solution:**

**Step 1:** Compute the mean/centroid of each class. In this example there are two classes called 'A' and 'B'.

Centroid of class 'A' =  $(3 + 5 + 4, 1 + 2 + 3)/3 = (12, 6)/3 = (4, 2)$

Centroid of class 'B' =  $(7 + 6 + 8, 6 + 7 + 5)/3 = (21, 18)/3 = (7, 6)$

Now given a test instance (6, 5), we can predict the class.

**Step 2:** Calculate the Euclidean distance between test instance (6, 5) and each of the centroid.

$$\text{Euc\_Dist}[(6, 5); (4, 2)] = \sqrt{(6-4)^2 + (5-2)^2} = \sqrt{13} = 3.6$$

$$\text{Euc\_Dist}[(6, 5); (7, 6)] = \sqrt{(6-7)^2 + (5-6)^2} = \sqrt{2} = 1.414$$

The test instance has smaller distance to class B. Hence, the class of this test instance is predicted as 'B'.

## 4.5 LOCALLY WEIGHTED REGRESSION (LWR)

Locally Weighted Regression (LWR) is a non-parametric supervised learning algorithm that performs local regression by combining regression model with nearest neighbor's model. LWR is also referred to as a memory-based method as it requires training data while prediction but uses only the training data instances locally around the point of interest. Using nearest neighbors algorithm, we find the instances that are closest to a test instance and fit linear function to each of those 'k' nearest instances in the local regression model. The key idea is that we need to approximate the linear functions of all 'k' neighbors that minimize the error such that the prediction line is no more linear but rather it is a curve.

Ordinary linear regression finds out a linear relationship between the input  $x$  and the output  $y$ .

Given training dataset  $T$ ,

Hypothesis function  $h_{\beta}(x)$ , the predicted target output is a linear function where  $\beta_0$  is the intercept and  $\beta_1$  is the coefficient of  $x$ .

It is given in Eq. (4.1) as,

$$h_{\beta}(x) = \beta_0 + \beta_1 x \quad (4.1)$$

The cost function is such that it minimizes the error difference between the predicted value  $h_{\beta}(x)$  and true value 'y' and it is given as in Eq. (4.2).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m (h_{\beta}(x_i) - y_i)^2 \quad (4.2)$$

where 'm' is the number of instances in the training dataset.

Now the cost function is modified for locally weighted linear regression including the weights only for the nearest neighbor points. Hence, the cost function is given as in Eq. (4.3).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m w_i (h_{\beta}(x_i) - y_i)^2 \quad (4.3)$$

where  $w_i$  is the weight associated with each  $x_i$ .

The weight function used is a Gaussian kernel that gives a higher value for instances that are close to the test instance, and for instances far away, it tends to zero but never equals to zero.

$w_i$  is computed in Eq. (4.4) as,

$$w_i = e^{-\frac{(x_i - x)^2}{2\tau^2}} \quad (4.4)$$

where,  $\tau$  is called the bandwidth parameter and controls the rate at which  $w_i$  reduces to zero with distance from  $x_i$ .

**Example 4.4:** Consider a simple example with four instances shown in Table 4.10 and apply locally weighted regression.

Table 4.10: Sample Table

| S.No. | Salary (in lakhs) | Expenditure (in thousands) |
|-------|-------------------|----------------------------|
| 1.    | 5                 | 25                         |
| 2.    | 1                 | 5                          |
| 3.    | 2                 | 7                          |
| 4.    | 1                 | 8                          |

**Solution:** Using linear regression model assuming we have computed the parameters:

$$\beta_0 = 4.72, \beta_1 = 0.62$$

Given a test instance with  $x = 2$ , the predicted  $y'$  is:

$$y' = \beta_0 + \beta_1 x = 4.72 + 0.62 \times 2 = 5.96$$

Applying the nearest neighbor model, we choose  $k = 3$  closest instances.

Table 4.11 shows the Euclidean distance calculation for the training instances.

Table 4.11: Euclidean Distance Calculation

| S.No. | $x$ = Salary (in lakhs) | $y$ = Expenditure (in thousands) | Euclidean Distance   |
|-------|-------------------------|----------------------------------|----------------------|
| 1.    | 5                       | 25                               | $\sqrt{(5-2)^2} = 3$ |
| 2.    | 1                       | 5                                | $\sqrt{(1-2)^2} = 1$ |
| 3.    | 2                       | 7                                | $\sqrt{(2-2)^2} = 0$ |
| 4.    | 1                       | 8                                | $\sqrt{(1-2)^2} = 1$ |

Instances 2, 3 and 4 are closer with smaller distances.

The mean value =  $(5 + 7 + 8)/3 = 20/3 = 6.67$ .

Using Eq. (4.4) compute the weights for the closest instances, using the Gaussian kernel,

$$w_i = e^{\frac{-(x_i - x)^2}{2\tau^2}}$$

Hence the weights of the closest instances is computed as follows,

Weight of Instance 2 is:

$$w_2 = e^{\frac{-(x_2 - x)^2}{2\tau^2}} = e^{\frac{-(1-2)^2}{2 \times 0.4^2}} = e^{-3.125} = 0.043$$

Weight of Instance 3 is:

$$w_3 = e^{\frac{-(x_3 - x)^2}{2\tau^2}} = e^{\frac{-(2-2)^2}{2 \times 0.4^2}} = e^0 = 1 \text{ [} w_3 \text{ is closer hence gets a higher weight value]}$$

Weight of Instance 4 is:

$$w_4 = e^{\frac{-(x_4 - x)^2}{2\sigma^2}} = e^{\frac{-(1-2)^2}{2 \times 0.4^2}} = e^{-3.125} = 0.043$$

The predicted output for the three closer instances is given as follows:

The predicted output of Instance 2 is:

$$y_2' = h_{\beta}(x_2) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 1 = 5.34$$

The predicted output of Instance 3 is:

$$y_3' = h_{\beta}(x_3) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 2 = 5.96$$

The predicted output of Instance 4 is:

$$y_4' = h_{\beta}(x_4) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 1 = 5.34$$

The error value is calculated as:

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m w_i (h_{\beta}(x_i) - y_i)^2 = \frac{1}{2} (0.043(5.34 - 5)^2 + 1(5.96 - 7)^2 + 0.043(5.34 - 8)^2) = 0.6953$$

Now, we need to adjust this cost function to minimize the error difference and get optimal  $\beta$  parameters.



## Chapter 5

# Regression Analysis

*"Regression analysis is the hydrogen bomb of the statistics arsenal."*  
— Charles Wheelan, *Naked Statistics: Stripping the Dread from the Data*

Regression analysis is a supervised learning method for predicting continuous variables. The difference between classification and regression analysis is that regression methods are used to predict qualitative variables or continuous numbers unlike categorical variables or labels. It is used to predict linear or non-linear relationships among variables of the given dataset. This chapter deals with an introduction of regression and its various types.

### Learning Objectives

- Understand the basics of regression analysis
- Introduce concepts of correlation and causation
- Learn about linear regression and its validation techniques
- Discuss about multiple linear regression
- Introduce logistic regression
- Study about the concept of regularization
- Study popular regression methods like Ridge, Lasso, and Elastic Net

## 5.1 INTRODUCTION TO REGRESSION

Regression analysis is the premier method of supervised learning. This is one of the most popular and oldest supervised learning technique. Given a training dataset  $D$  containing  $N$  training points  $(x_i, y_i)$ , where  $i = 1 \dots N$ , regression analysis is used to model the relationship between one or more independent variables  $x_i$  and a dependent variable  $y_i$ . The relationship between the dependent and independent variables can be represented as a function as follows:

$$y = f(x) \quad (5.1)$$

Here, the feature variable  $x$  is also known as an explanatory variable, exploratory variable, a predictor variable, an independent variable, a covariate, or a domain point.  $y$  is a dependent variable. Dependent variables are also called as labels, target variables, or response variables.

Regression analysis determines the change in response variables when one exploration variable is varied while keeping all other parameters constant. This is used to determine the relationship each of the exploratory variables exhibits. Thus, regression analysis is used for prediction and forecasting.

Regression is used to predict continuous variables or quantitative variables such as price and revenue. Thus, the primary concern of regression analysis is to find answer to questions such as:

1. What is the relationship between the variables?
2. What is the strength of the relationships?
3. What is the nature of the relationship such as linear or non-linear?
4. What is the relevance of the attributes?
5. What is the contribution of each attribute?

There are many applications of regression analysis. Some of the applications of regressions include predicting:

1. Sales of a goods or services
2. Value of bonds in portfolio management
3. Premium on insurance companies
4. Yield of crops in agriculture
5. Prices of real estate

## 5.2 INTRODUCTION TO LINEARITY, CORRELATION, AND CAUSATION

The quality of the regression analysis is determined by the factors such as correlation and causation.

### *Regression and Correlation*

Correlation among two variables can be done effectively using a Scatter plot, which is a plot between explanatory variables and response variables. It is a 2D graph showing the relationship between two variables. The x-axis of the scatter plot is independent, or input or predictor variables and y-axis of the scatter plot is output or dependent or predicted variables. The scatter plot is useful in exploring data. Some of the scatter plots are shown in Figure 5.1. The Pearson correlation coefficient is the most common test for determining correlation if there is an association between two variables. The correlation coefficient is denoted by  $r$ . Correlation is discussed in Chapter 2 of this book. The positive, negative, and random correlations are given in Figure 5.1. In positive correlation, one variable change is associated with the change in another variable. In negative correlation, the relationship between the variables is reciprocal while in random correlation, no relationship exists between variables.

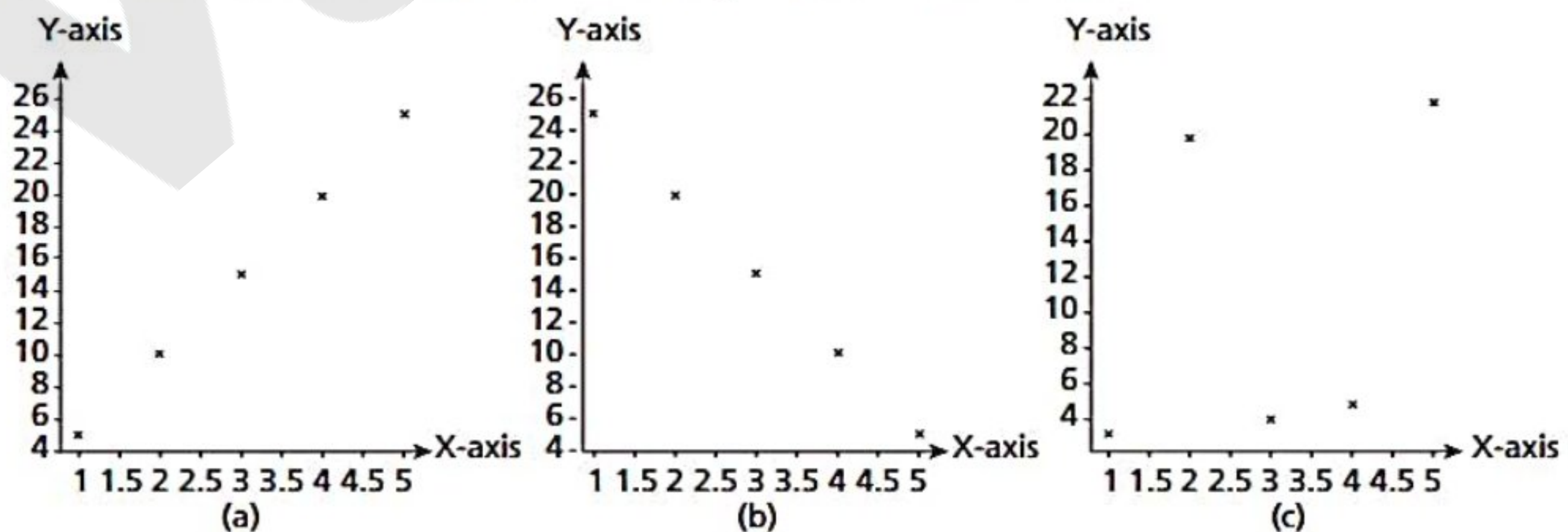


Figure 5.1: Examples of (a) Positive Correlation (b) Negative Correlation (c) Random Points with No Correlation

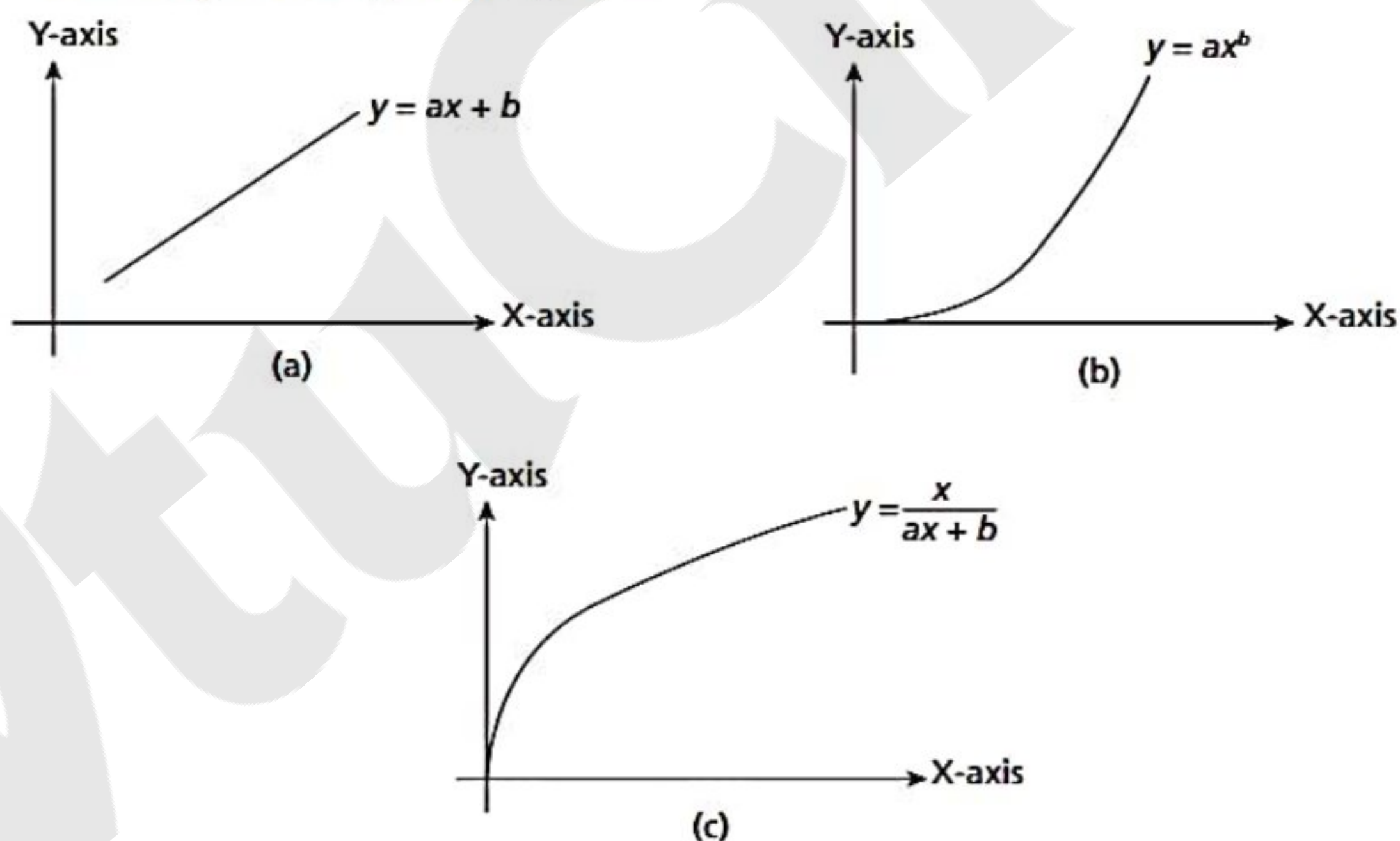
While correlation is about relationships among variables, say  $x$  and  $y$ , regression is about predicting one variable given another variable.

### Regression and Causation

Causation is about causal relationship among variables, say  $x$  and  $y$ . Causation means knowing whether  $x$  causes  $y$  to happen or vice versa.  $x$  causes  $y$  is often denoted as  $x$  implies  $y$ . Correlation and Regression relationships are not same as causation relationship. For example, the correlation between economical background and marks scored does not imply that economic background causes high marks. Similarly, the relationship between higher sales of cool drinks due to a rise in temperature is not a causal relation. Even though high temperature is the cause of cool drinks sales, it depends on other factors too.

### Linearity and Non-linearity Relationships

The linearity relationship between the variables means the relationship between the dependent and independent variables can be visualized as a straight line. The line of the form,  $y = ax + b$  can be fitted to the data points that indicate the relationship between  $x$  and  $y$ . By linearity, it is meant that as one variable increases, the corresponding variable also increases in a linear manner. A linear relationship is shown in Figure 5.2 (a). A non-linear relationship exists in functions such as exponential function and power function and it is shown in Figures 5.2 (b) and 5.2 (c). Here,  $x$ -axis is given by  $x$  data and  $y$ -axis is given by  $y$  data.



**Figure 5.2:** (a) Example of Linear Relationship of the Form  $y = ax + b$  (b) Example of a Non-linear Relationship of the Form  $y = ax^b$  (c) Examples of a Non-linear Relationship  $y = \frac{x}{ax + b}$

The functions like exponential function ( $y = ax^b$ ) and power function  $\left(y = \frac{x}{ax + b}\right)$  are non-linear relationships between the dependent and independent variables that cannot be fitted in a line. This is shown in Figures 5.2 (b) and (c).

## Types of Regression Methods

The classification of regression methods is shown in Figure 5.3.

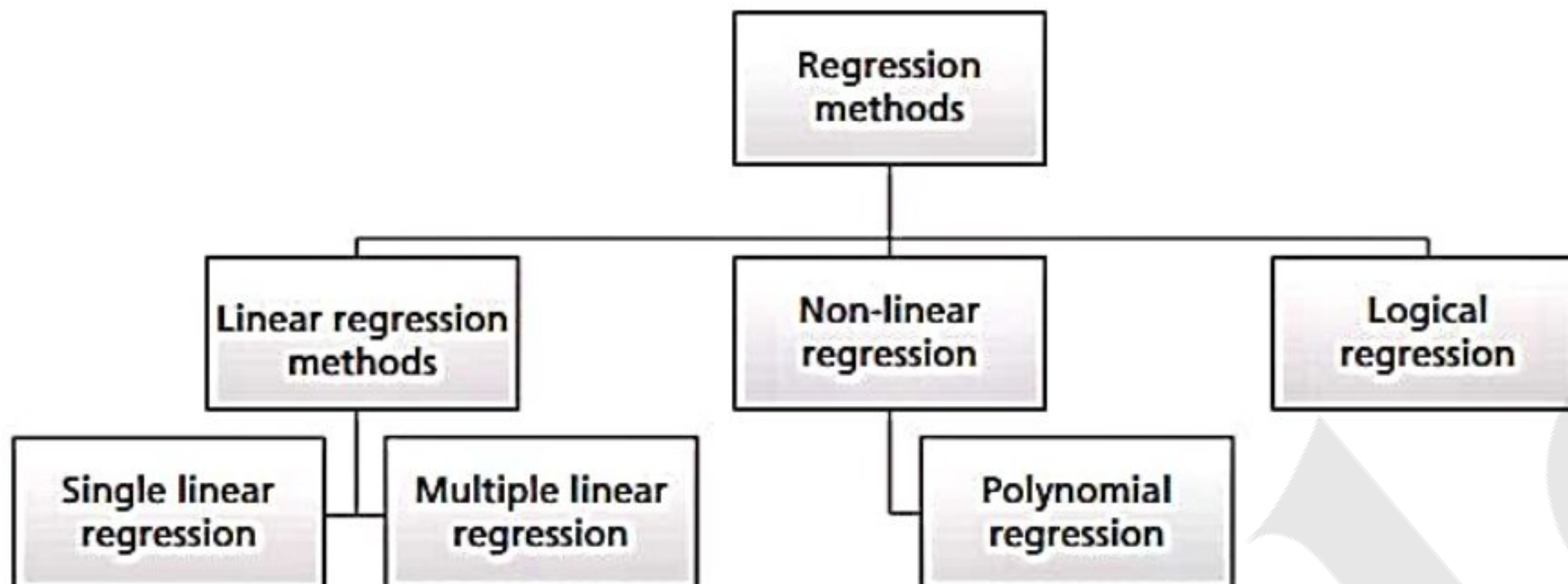


Figure 5.3: Types of Regression Methods

**Linear Regression** It is a type of regression where a line is fitted upon given data for finding the linear relationship between one independent variable and one dependent variable to describe relationships.

**Multiple Regression** It is a type of regression where a line is fitted for finding the linear relationship between two or more independent variables and one dependent variable to describe relationships among variables.

**Polynomial Regression** It is a type of non-linear regression method of describing relationships among variables where  $N^{\text{th}}$  degree polynomial is used to model the relationship between one independent variable and one dependent variable. Polynomial multiple regression is used to model two or more independent variables and one dependant variable.

**Logistic Regression** It is used for predicting categorical variables that involve one or more independent variables and one dependent variable. This is also known as a binary classifier.

**Lasso and Ridge Regression Methods** These are special variants of regression method where regularization methods are used to limit the number and size of coefficients of the independent variables.

## Limitations of Regression Method

1. **Outliers** – Outliers are abnormal data. It can bias the outcome of the regression model, as outliers push the regression line towards it.
2. **Number of cases** – The ratio of independent and dependent variables should be at least 20 : 1. For every explanatory variable, there should be at least 20 samples. Atleast five samples are required in extreme cases.
3. **Missing data** – Missing data in training data can make the model unfit for the sampled data.
4. **Multicollinearity** – If exploratory variables are highly correlated (0.9 and above), the regression is vulnerable to bias. Singularity leads to perfect correlation of 1. The remedy is to remove exploratory variables that exhibit correlation more than 1. If there is a tie, then the tolerance ( $1 - R^2$ ) is used to eliminate variables that have the greatest value.

### 5.3 INTRODUCTION TO LINEAR REGRESSION

In the simplest form, the linear regression model can be created by fitting a line among the scattered data points. The line is of the form given in Eq. (5.2).

$$y = a_0 + a_1 \times x + e \quad (5.2)$$

Here,  $a_0$  is the intercept which represents the bias and  $a_1$  represents the slope of the line. These are called regression coefficients.  $e$  is the error in prediction.

The assumptions of linear regression are listed as follows:

1. The observations ( $y$ ) are random and are mutually independent.
2. The difference between the predicted and true values is called an error. The error is also mutually independent with the same distributions such as normal distribution with zero mean and constant variables.
3. The distribution of the error term is independent of the joint distribution of explanatory variables.
4. The unknown parameters of the regression models are constants.

The idea of linear regression is based on Ordinary Least Square (OLS) approach. This method is also known as ordinary least squares method. In this method, the data points are modelled using a straight line. Any arbitrarily drawn line is not an optimal line. In Figure 5.4, three data points and their errors ( $e_1, e_2, e_3$ ) are shown. The vertical distance between each point and the line (predicted by the approximate line equation  $y = a_0 + a_1x$ ) is called an error. These individual errors are added to compute the total error of the predicted line. This is called sum of residuals. The squares of the individual errors can also be computed and added to give a sum of squared error. The line with the lowest sum of squared error is called line of best fit.

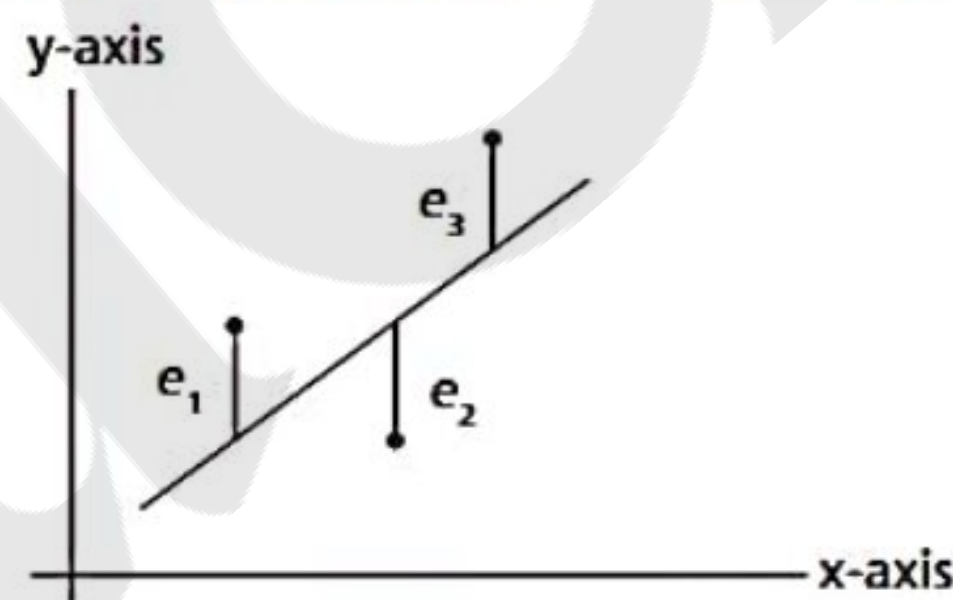


Figure 5.4: Data Points and their Errors

In another words, OLS is an optimization technique where the difference between the data points and the line is optimized.

Mathematically, based on Eq. (5.2), the line equations for points ( $x_1, x_2, \dots, x_n$ ) are:

$$\begin{aligned} y_1 &= (a_0 + a_1x_1) + e_1 \\ y_2 &= (a_0 + a_1x_2) + e_2 \\ &\vdots \\ &\vdots \\ &\vdots \\ y_n &= (a_0 + a_1x_n) + e_n \end{aligned} \quad (5.3)$$

$$\text{In general, the error is given as: } e_i = y_i - (a_0 + a_1x_i) \quad (5.4)$$

This can be extended into the set of equations as shown in Eq. (5.3).

Here, the terms  $(e_1, e_2, \dots, e_n)$  are error associated with the data points and denote the difference between the true value of the observation and the point on the line. This is also called as residuals. The residuals can be positive, negative or zero.

A regression line is the line of best fit for which the sum of the squares of residuals is minimum. The minimization can be done as minimization of individual errors by finding the parameters  $a_0$  and  $a_1$  such that:

$$E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_1 x_i)) \quad (5.5)$$

Or as the minimization of sum of absolute values of the individual errors:

$$E = \sum_{i=1}^n |e_i| = \sum_{i=1}^n |y_i - (a_0 + a_1 x_i)| \quad (5.6)$$

Or as the minimization of the sum of the squares of the individual errors:

$$E = \sum_{i=1}^n (e_i)^2 = \sum_{i=1}^n (y_i - (a_0 + a_1 x_i))^2 \quad (5.7)$$

Sum of the squares of the individual errors, often preferred as individual errors (positive and negative errors), do not get cancelled out and are always positive, and sum of squares results in a large increase even for a small change in the error. Therefore, this is preferred for linear regression.

Therefore, linear regression is modelled as a minimization function as follows:

$$\begin{aligned} J(a_1, a_0) &= \sum_{i=1}^n [y_i - f(x_i)]^2 \\ &= \sum_{i=1}^n [y_i - (a_0 + a_1 x_i)]^2 \end{aligned} \quad (5.8)$$

Here,  $J(a_0, a_1)$  is the criterion function of parameters  $a_0$  and  $a_1$ . This needs to be minimized. This is done by differentiating and substituting to zero. This yields the coefficient values of  $a_0$  and  $a_1$ . The values of estimates of  $a_0$  and  $a_1$  are given as follows:

$$a_1 = \frac{(\overline{xy}) - (\bar{x})(\bar{y})}{(\overline{x^2}) - (\bar{x})^2} \quad (5.9)$$

And the value of  $a_0$  is given as follows:

$$a_0 = (\bar{y}) - a_1 \times \bar{x} \quad (5.10)$$

Let us consider a simple problem to illustrate the usage of the above concept.

**Example 5.1:** Let us consider an example where the five weeks' sales data (in Thousands) is given as shown below in Table 5.1. Apply linear regression technique to predict the 7<sup>th</sup> and 9<sup>th</sup> month sales.

Table 5.1: Sample Data

| $x_i$<br>(Week) | $y_i$<br>(Sales in Thousands) |
|-----------------|-------------------------------|
| 1               | 1.2                           |
| 2               | 1.8                           |
| 3               | 2.6                           |
| 4               | 3.2                           |
| 5               | 3.8                           |

**Solution:** Here, there are 5 items, i.e.,  $i = 1, 2, 3, 4, 5$ . The computation table is shown below (Table 5.2). Here, there are five samples, so  $i$  ranges from 1 to 5.

**Table 5.2: Computation Table**

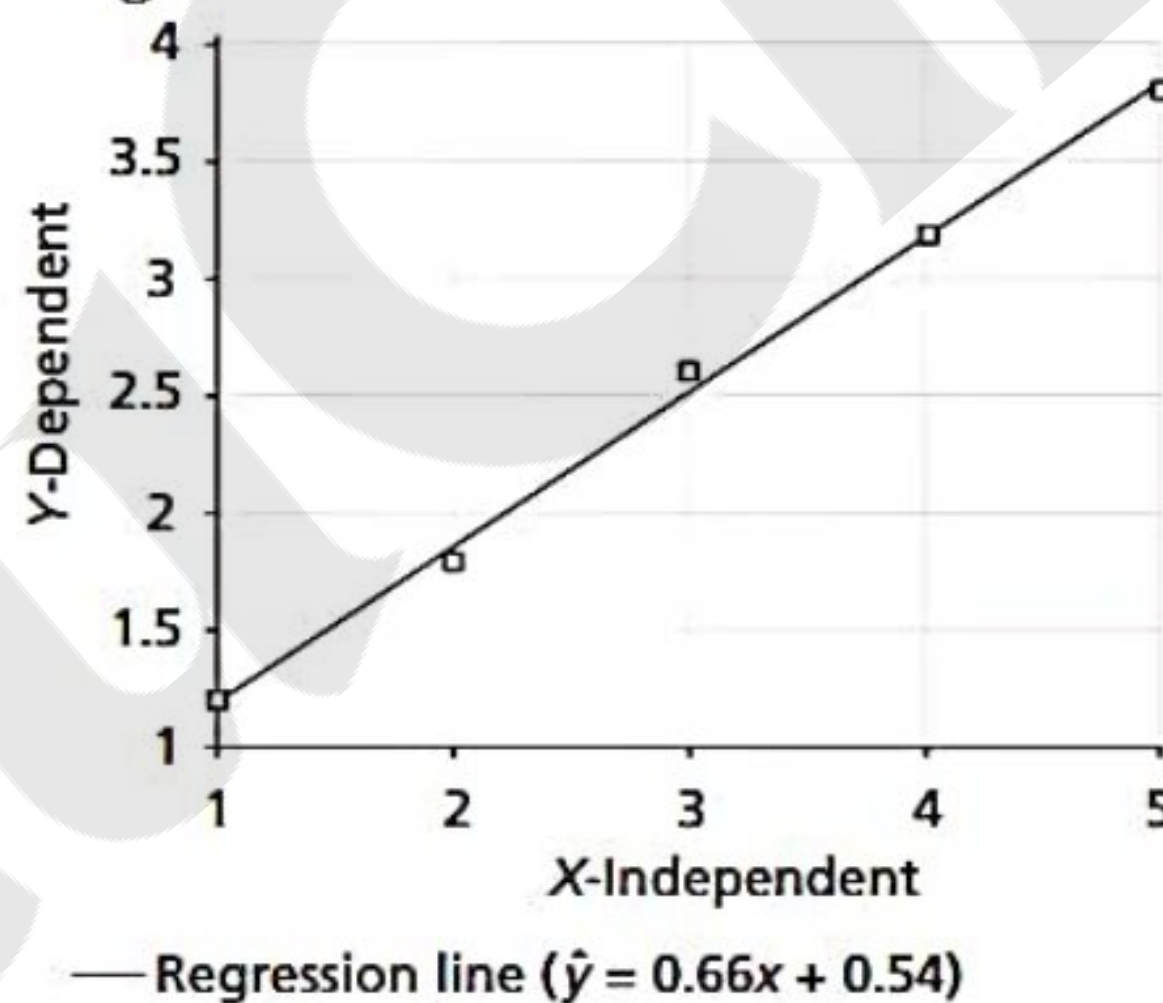
| $x_i$                                                                 | $y_i$                                                                        | $(x_i)^2$                                                                         | $x_i \times y_i$                                                                              |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1                                                                     | 1.2                                                                          | 1                                                                                 | 1.2                                                                                           |
| 2                                                                     | 1.8                                                                          | 4                                                                                 | 3.6                                                                                           |
| 3                                                                     | 2.6                                                                          | 9                                                                                 | 7.8                                                                                           |
| 4                                                                     | 3.2                                                                          | 16                                                                                | 12.8                                                                                          |
| 5                                                                     | 3.8                                                                          | 25                                                                                | 19                                                                                            |
| Sum = 15<br>Average of $(x_i)$<br>$= \bar{x} = \frac{15}{5}$<br>$= 3$ | Sum = 12.6<br>Average of $(y_i)$<br>$= \bar{y} = \frac{12.6}{5}$<br>$= 2.52$ | Sum = 55<br>Average of $(x_i^2)$<br>$= \overline{x_i^2} = \frac{55}{5}$<br>$= 11$ | Sum = 44.4<br>Average of $(x_i \times y_i)$<br>$= \overline{xy} = \frac{44.4}{5}$<br>$= 8.88$ |

Let us compute the slope and intercept now using Eq. (5.9) as:

$$a_1 = \frac{8.88 - 3(2.52)}{11 - 3^2} = 0.66$$

$$a_0 = 2.52 - 0.66 \times 3 = 0.54$$

The fitted line is shown in Figure 5.5.



**Figure 5.5: Linear Regression Model Constructed**

Let us model the relationship as  $y = a_0 + a_1 \times x$ . Therefore, the fitted line for the above data is:  $y = 0.54 + 0.66 \times x$ .

The predicted 7<sup>th</sup> week sale would be (when  $x = 7$ ),  $y = 0.54 + 0.66 \times 7 = 5.16$  and the 12<sup>th</sup> month,  $y = 0.54 + 0.66 \times 12 = 8.46$ . All sales are in thousands.

Scan for 'Additional Examples'



### Linear Regression in Matrix Form

Matrix notations can be used for representing the values of independent and dependent variables. This is illustrated through Example 5.2.

The Eq. (5.3) can be written in the form of matrix as follows:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix} \quad (5.11)$$

This can be written as:

$Y = Xa + e$ , where  $X$  is an  $n \times 2$  matrix,  $Y$  is an  $n \times 1$  vector,  $a$  is a  $2 \times 1$  column vector and  $e$  is an  $n \times 1$  column vector.

**Example 5.2:** Find linear regression of the data of week and product sales (in Thousands) given in Table 5.3. Use linear regression in matrix form.

Table 5.3: Sample Data for Regression

| $x_i$<br>(Week) | $y_i$<br>(Product Sales in Thousands) |
|-----------------|---------------------------------------|
| 1               | 1                                     |
| 2               | 3                                     |
| 3               | 4                                     |
| 4               | 8                                     |

**Solution:** Here, the dependent variable  $X$  is be given as:

$$x^T = [1 \ 2 \ 3 \ 4]$$

And the independent variable is given as follows:

$$y^T = [1 \ 3 \ 4 \ 8]$$

The data can be given in matrix form as follows:

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}. \text{ The first column can be used for setting bias.}$$

and  $Y = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix}$

The regression is given as:

$$a = ((X^T X)^{-1} X^T) Y$$

The computation order of this equation is shown step by step as:

$$1. \text{ Computation of } (X^T X) = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} \times \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}$$

$$2. \text{ Computation of matrix inverse of } (X^T X)^{-1} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}^{-1} = \begin{pmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{pmatrix}$$

$$3. \text{ Computation of } ((X^T X)^{-1} X^T) = \begin{pmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{pmatrix}$$

$$4. \text{ Finally, } ((X^T X)^{-1} X^T) Y = \begin{pmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{pmatrix} \times \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix} = \begin{pmatrix} -1.5 \\ 2.2 \end{pmatrix} \begin{pmatrix} \text{Intercept} \\ \text{slope} \end{pmatrix}$$

Thus, the substitution of values in Eq. (5.11) using the previous steps yields the fitted line as  $2.2x - 1.5$ .

## 5.5 MULTIPLE LINEAR REGRESSION

Multiple regression model involves multiple predictors or independent variables and one dependent variable. This is an extension of the linear regression problem. The basic assumptions of multiple linear regression are that the independent variables are not highly correlated and hence multicollinearity problem does not exist. Also, it is assumed that the residuals are normally distributed.

For example, the multiple regression of two variables  $x_1$  and  $x_2$  is given as follows:

$$\begin{aligned} y &= f(x_1, x_2) \\ &= a_0 + a_1x_1 + a_2x_2 \end{aligned} \quad (5.21)$$

In general, this is given for ' $n$ ' independent variables as:

$$\begin{aligned} y &= f(x_1, x_2, x_3, \dots, x_n) \\ &= a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n + \varepsilon \end{aligned} \quad (5.22)$$

Here,  $(x_1, x_2, \dots, x_n)$  are predictor variables,  $y$  is the dependent variable,  $(a_0, a_1, \dots, a_n)$  are the coefficients of the regression equation and  $\varepsilon$  is the error term. This is illustrated through Example 5.5.

**Example 5.5:** Apply multiple regression for the values given in Table 5.7 where weekly sales along with sales for products  $x_1$  and  $x_2$  are provided. Use matrix approach for finding multiple regression.

Table 5.7: Sample Data

| $x_1$<br>(Product One Sales) | $x_2$<br>(Product Two Sales) | $y$<br>Output Weekly Sales (in Thousands) |
|------------------------------|------------------------------|-------------------------------------------|
| 1                            | 4                            | 1                                         |
| 2                            | 5                            | 6                                         |
| 3                            | 8                            | 8                                         |
| 4                            | 2                            | 12                                        |

**Solution:** Here, the matrices for  $Y$  and  $X$  are given as follows:

$$X = \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \text{ and } Y = \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix}.$$

The coefficient of the multiple regression equation is given as  $a = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix}$ .

The regression coefficient for multiple regression is calculated the same way as linear regression:

$$\hat{a} = ((X^T X)^{-1} X^T) Y \quad (5.23)$$

Using Eq. (5.23), and substituting the values (Similar to Problem 5.2), one gets  $\hat{a}$  as:

$$\begin{aligned} &= \left( \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \right)^{-1} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} \\ &= \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix} \end{aligned}$$

Here, the coefficients are  $a_0 = -1.69$ ,  $a_1 = 3.48$  and  $a_2 = -0.05$ . Hence, the constructed model is:  
 $y = -1.69 + 3.48x_1 - 0.05x_2$

## 5.6 POLYNOMIAL REGRESSION

If the relationship between the independent and dependent variables is not linear, then linear regression cannot be used as it will result in large errors. The problem of non-linear regression can be solved by two methods:

1. Transformation of non-linear data to linear data, so that the linear regression can handle the data
2. Using polynomial regression

### Transformations

The first method is called transformation. The trick is to convert non-linear data to linear data that can be handled using the linear regression method. Let us consider an exponential function  $y = ae^{bx}$ . The transformation can be done by applying log function to both sides to get:

$$\ln y = bx + \ln a \quad (5.24)$$

Similarly, power function of the form ( $y = ax^b$ ) can be transformed by applying log function on both sides as follows:

$$\log_{10} y = b \log_{10} x + \log_{10} a \quad (5.25)$$

Once the transformation is carried out, linear regression can be performed and after the results are obtained, the inverse functions can be applied to get the desired result.

### Polynomial Regression

It can handle non-linear relationships among variables by using  $n^{\text{th}}$  degree of a polynomial. Instead of applying transforms, polynomial regression can be directly used to deal with different levels of curvilinearity.

Polynomial regression provides a non-linear curve such as quadratic and cubic. For example, the second-degree transformation (called quadratic transformation) is given as:  $y = a_0 + a_1x + a_2x^2$  and the third-degree polynomial is called cubic transformation given as:  $y = a_0 + a_1x + a_2x^2 + a_3x^3$ . Generally, polynomials of maximum degree 4 are used, as higher order polynomials take some strange shapes and make the curve more flexible. It leads to a situation of overfitting and hence is avoided.

Let us consider a polynomial of 2<sup>nd</sup> degree. Given points  $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ , the objective is to fit a polynomial of degree 2. The polynomial of degree 2 is given as:

$$y = a_0 + a_1x + a_2x^2 \quad (5.26)$$

Such that the error  $E = \sum_{i=1}^n [y_i - (a_0 + a_1x_i + a_2x_i^2)]^2$  is minimized. The coefficients  $a_0, a_1, a_2$  of Eq. (5.26) can be obtained by taking partial derivatives with respect to each of the coefficients as  $\frac{\partial E}{\partial a_0}, \frac{\partial E}{\partial a_1}, \frac{\partial E}{\partial a_2}$  and substituting it with zero. This results in 2 + 1 equations given as follows:

$$\begin{aligned} na_0 + \left( \sum_{i=1}^n x_i \right) a_1 + \left( \sum_{i=1}^n x_i^2 \right) a_2 &= \sum_{i=1}^n y_i \\ \left( \sum_{i=1}^n x_i \right) a_0 + \left( \sum_{i=1}^n x_i^2 \right) a_1 + \left( \sum_{i=1}^n x_i^3 \right) a_2 &= \sum_{i=1}^n x_i y_i \\ \left( \sum_{i=1}^n x_i^2 \right) a_0 + \left( \sum_{i=1}^n x_i^3 \right) a_1 + \left( \sum_{i=1}^n x_i^4 \right) a_2 &= \sum_{i=1}^n x_i^2 y_i \end{aligned} \quad (5.27)$$

The best line is the line that minimizes the error between line and data points. Arranging the coefficients of the above equation in the matrix form results in:

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum (x_i y_i) \\ \sum (x_i^2 y_i) \end{bmatrix} \quad (5.28)$$

This is of the form  $Xa = B$ . One can solve this equation for  $a$  as:

$$a = X^{-1}B \quad (5.29)$$

**Example 5.6:** Consider the data provided in Table 5.8 and fit it using the second-order polynomial.

Table 5.8: Sample Data

| $x$ | $y$ |
|-----|-----|
| 1   | 1   |
| 2   | 4   |
| 3   | 9   |
| 4   | 15  |

**Solution:** For applying polynomial regression, computation is done as shown in Table 5.9.

Here, the order is 2 and the sample  $i$  ranges from 1 to 4.

Table 5.9: Computation Table

| $x_i$           | $y_i$           | $x_i y_i$           | $x_i^2$           | $x_i^2 y_i$            | $x_i^3$            | $x_i^4$            |
|-----------------|-----------------|---------------------|-------------------|------------------------|--------------------|--------------------|
| 1               | 1               | 1                   | 1                 | 1                      | 1                  | 1                  |
| 2               | 4               | 8                   | 4                 | 16                     | 8                  | 16                 |
| 3               | 9               | 27                  | 9                 | 81                     | 27                 | 81                 |
| 4               | 15              | 60                  | 16                | 240                    | 64                 | 256                |
| $\sum x_i = 10$ | $\sum y_i = 29$ | $\sum x_i y_i = 96$ | $\sum x_i^2 = 30$ | $\sum x_i^2 y_i = 338$ | $\sum x_i^3 = 100$ | $\sum x_i^4 = 354$ |

It can be noted that,  $N = 4$ ,  $\sum y_i = 29$ ,  $\sum x_i y_i = 96$ ,  $\sum x_i^2 y_i = 338$ . When the order is 2, the matrix using Eq. (5.28) is given as follows:

$$\begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 29 \\ 96 \\ 338 \end{bmatrix}$$

Therefore, using Eq. (5.29), one can get coefficients as:

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix}^{-1} \times \begin{bmatrix} 29 \\ 96 \\ 338 \end{bmatrix} = \begin{pmatrix} -0.75 \\ 0.95 \\ 0.75 \end{pmatrix}.$$

This leads to the regression equation using Eq. (5.26) as:

$$y = -0.75 + 0.95x + 0.75x^2$$

## 5.7 LOGISTIC REGRESSION

Linear regression predicts the numerical response but is not suitable for predicting the categorical variables. When categorical variables are involved, it is called classification problem. Logistic regression is suitable for binary classification problem. Here, the output is often a categorical variable. For example, the following scenarios are instances of predicting categorical variables.

1. Is the mail spam or not spam? The answer is yes or no. Thus, categorical dependant variable is a binary response of yes or no.
2. If the student should be admitted or not is based on entrance examination marks. Here, categorical variable response is admitted or not.
3. The student being pass or fail is based on marks secured.

Thus, logistic regression is used as a binary classifier and works by predicting the probability of the categorical variable. In general, it takes one or more features  $x$  and predicts the response  $y$ . If the probability is predicted via linear regression, it is given as:

$$p(x) = a_0 + a_1x$$

Hence, logistic regression tries to model the probability of the particular response variable. In email classification problem, say normal email or spam, if the probability of the response variable is 0.7, then there is a 70% possibility of a normal mail.

Linear regression generated value is in the range  $-\infty$  to  $+\infty$ , whereas the probability of the response variable ranges between 0 and 1. Hence, there must be a mapping function to map the value  $-\infty$  to  $+\infty$  to 0–1. The core of the mapping function in logistic regression method is sigmoidal function. A sigmoidal function is a 'S' shaped function that yields values between 0 and 1. This is known as logit function. This is mathematically represented as:

$$\text{logit}(x) = \frac{1}{1 + e^{-x}} \quad (5.30)$$

Here,  $x$  is the independent variable and  $e$  is the Euler number. The purpose of the logit function is to map any real number to zero or 1.

Logistic regression can be viewed as an extension of linear regression, but the only difference is that the output of linear regression can be an extremely high number. This needs to be mapped into the range 0–1, as probability can have values only in the range 0–1. This problem is solved using log odd or logit functions. What is the difference between odds and probability? Odds and probability (or likelihood) are two sides of a coin and represent uncertainty. The odds are defined as the ratio of the probability of an event and probability of an event that is not happening. This is given as:

$$\text{odd} = \frac{\text{probability of an event}}{\text{probability of an non-event}} = \frac{p}{1-p} \quad (5.31)$$

Log-odds can be taken for the odds, resulting in:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = a_0 + a_1x \quad (5.32)$$

Here,  $\log(.)$  is a logit function or log odds function. One can solve for  $p(x)$  by taking the inverse of the above function as:

$$p(x) = \frac{\exp(a_0 + a_1x)}{1 + \exp(a_0 + a_1x)} \quad (5.33)$$

This is the same sigmoidal function. It always gives the value in the range 0–1. Dividing the numerator and denominator by the denominator, one gets:

$$p(x) = \frac{1}{1 + \exp(-a_0 - a_1x)} \quad (5.34)$$

One can rearrange this by taking the minus sign outside to get the following logistic function:

$$p(x) = \frac{1}{1 + \exp(-(a_0 + a_1x))} \quad (5.35)$$

Here,  $x$  is the explanatory or predictor variable,  $e$  is the Euler number, and  $a_0, a_1$  are the regression coefficients. The coefficients  $a_0, a_1$  can be learned and the predictor predicts  $p(x)$  directly using the threshold function as:

$$y = \begin{cases} 1 & \text{if } p(x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad (5.36)$$

**Example 5.7:** Let us assume a binomial logistic regression problem where the classes are pass and fail. The student dataset has entrance mark based on the historic data of those who are selected or not selected. Based on the logistic regression, the values of the learnt parameters are  $a_0 = 1$  and  $a_1 = 8$ . Assuming marks of  $x = 60$ , compute the resultant class.

**Solution:** The values of regression coefficients are  $a_0 = 1$  and  $a_1 = 8$ , and given that  $x = 60$ .

Based on the regression coefficients,  $z$  can be computed as:

$$\begin{aligned} z &= a_0 + a_1 x \\ &= 1 + 8 \times 60 = 481 \end{aligned}$$

One can fit this in a sigmoidal function using Eq. (5.30) to get the probability as:

$$y = \frac{1}{1 + \exp(-481)} = \frac{1}{2.271} = 0.44$$

If we assume the threshold value as 0.5, then it is observed that  $0.44 < 0.5$ , therefore, the candidate with marks 60 is not selected.

To determine the relationship between dependant and independent variables, parameters need to be obtained. In logistic regression, the parameters are obtained through maximum likelihood function (MLE) using the training data. The aim is to learn the values of parameters of the logistic model ( $a$ 's) by minimizing the error in the probability predicted by the model.

There can be many different sets of coefficients available. The optimal value of the parameters is obtained by using the MLE function, which is a set of coefficients for which the probability of getting the observed data is maximum.

If  $\pi$  is the success of the outcome and  $1 - \pi$  is the failure of the outcome, then the likelihood function is given as:

$$L(a : y) = \prod_{i=1}^n \left( \frac{\pi_i}{1 - \pi_i} \right)^{y_i} (1 - \pi_i) \quad (5.37)$$

To determine the value of the parameters, the log of the likelihood function is taken. Techniques like Newton method can be used to maximize the log-likelihood of the function.

Logistic regression is suitable for binary classification. The idea can be extended for multiple classes called multinomial logistic regression. Let us assume that there are three classes 1, 2 and 3. Then, the multinomial logistic regression creates three classification problems – class 1 and Not class 1, class 2 and Not class 2, and finally class 3 and Not class 3. Three problems are simultaneously used to find the maximum probability relative to others to get the appropriate class.

Logistic regression is a simple and efficient method for binary classification. The model can be easily interpreted too. The disadvantages of logistic regression are that multinomial logistic regression cannot handle many attributes and can handle only linear features. Also, if all the attributes have multicollinearity problem, the logistic method does not work effectively.



## Chapter 6

# Decision Tree Learning

*"Prediction is very difficult, especially if it's about the future."*

— Niels Bohr

*Decision Tree Learning* is a widely used predictive model for supervised learning that spans over a number of practical applications in various areas. It is used for both classification and regression tasks. The decision tree model basically represents logical rules that predict the value of a target variable by inferring from data features. This chapter provides a keen insight into how to construct a decision tree and infer knowledge from the tree.

### Learning Objectives

- Understand the structure of the decision tree
- Know about the fundamentals of Entropy
- Learn and understand popular univariate Decision Tree Induction algorithms such as ID3, C4.5, and multivariate decision tree algorithm such as CART
- Deal with continuous attributes using improved C4.5 algorithm
- Construct Classification and Regression Tree (CART) for classifying both categorical and continuous-valued target variables
- Construct regression trees where the target feature is a continuous-valued variable
- Understand the basics of validating and pruning of decision trees

## 6.1 INTRODUCTION TO DECISION TREE LEARNING MODEL

Decision tree learning model, one of the most popular supervised predictive learning models, classifies data instances with high accuracy and consistency. The model performs an *inductive inference* that reaches a general conclusion from observed examples. This model is variably used for solving complex classification applications.

Decision tree is a concept tree which summarizes the information contained in the training dataset in the form of a tree structure. Once the concept model is built, test data can be easily classified.

This model can be used to classify both categorical target variables and continuous-valued target variables. Given a training dataset  $X$ , this model computes a hypothesis function  $f(X)$  as decision tree.

Inputs to the model are data instances or objects with a set of features or attributes which can be discrete or continuous and the output of the model is a decision tree which predicts or classifies the target class for the test data object.

In statistical terms, attributes or features are called as independent variables. The target feature or target class is called as response variable which indicates the category we need to predict on a test object.

The decision tree learning model generates a complete hypothesis space in the form of a tree structure with the given training dataset and allows us to search through the possible set of hypotheses which in fact would be a smaller decision tree as we walk through the tree. This kind of search bias is called as preference bias.

### 6.1.1 Structure of a Decision Tree

A decision tree has a structure that consists of a root node, internal nodes/decision nodes, branches, and terminal nodes/leaf nodes. The topmost node in the tree is the root node. Internal nodes are the test nodes and are also called as decision nodes. These nodes represent a choice or test of an input attribute and the outcome or outputs of the test condition are the branches emanating from this decision node. The branches are labelled as per the outcomes or output values of the test condition. Each branch represents a sub-tree or subsection of the entire tree. Every decision node is part of a path to a leaf node. The leaf nodes represent the labels or the outcome of a decision path. The labels of the leaf nodes are the different target classes a data instance can belong to.

Every path from root to a leaf node represents a logical rule that corresponds to a conjunction of test attributes and the whole tree represents a disjunction of these conjunctions. The decision tree model, in general, represents a collection of logical rules of classification in the form of a tree structure.

Decision networks, otherwise called as influence diagrams, have a directed graph structure with nodes and links. It is an extension of Bayesian belief networks that represents information about each node's current state, its possible actions, the possible outcome of those actions, and their utility. The concept of Bayesian Belief Network (BBN) is discussed in Chapter 9.

Figure 6.1 shows symbols that are used in this book to represent different nodes in the construction of a decision tree. A circle is used to represent a root node, a diamond symbol is used to represent a decision node or the internal nodes, and all leaf nodes are represented with a rectangle.

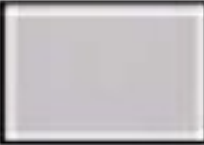
|                                                                                     |               |
|-------------------------------------------------------------------------------------|---------------|
|  | Root note     |
|  | Decision node |
|  | Leaf node     |

Figure 6.1: Nodes in a Decision Tree

A decision tree consists of two major procedures discussed below.

## ***Building the Tree***

**Goal** Construct a decision tree with the given training dataset. The tree is constructed in a top-down fashion. It starts from the root node. At every level of tree construction, we need to find the best split attribute or best decision node among all attributes. This process is recursive and continued until we end up in the last level of the tree or finding a leaf node which cannot be split further. The tree construction is complete when all the test conditions lead to a leaf node. The leaf node contains the target class or output of classification.

**Output** Decision tree representing the complete hypothesis space.

## ***Knowledge Inference or Classification***

**Goal** Given a test instance, infer to the target class it belongs to.

**Classification** Inferring the target class for the test instance or object is based on inductive inference on the constructed decision tree. In order to classify an object, we need to start traversing the tree from the root. We traverse as we evaluate the test condition on every decision node with the test object attribute value and walk to the branch corresponding to the test's outcome. This process is repeated until we end up in a leaf node which contains the target class of the test object.

**Output** Target label of the test instance.

## ***Advantages of Decision Trees***

1. Easy to model and interpret
2. Simple to understand
3. The input and output attributes can be discrete or continuous predictor variables.
4. Can model a high degree of nonlinearity in the relationship between the target variables and the predictor variables
5. Quick to train

## ***Disadvantages of Decision Trees***

Some of the issues that generally arise with a decision tree learning are that:

1. It is difficult to determine how deeply a decision tree can be grown or when to stop growing it.
2. If training data has errors or missing attribute values, then the decision tree constructed may become unstable or biased.
3. If the training data has continuous valued attributes, handling it is computationally complex and has to be discretized.
4. A complex decision tree may also be over-fitting with the training data.
5. Decision tree learning is not well suited for classifying multiple output classes.
6. Learning an optimal decision tree is also known to be NP-complete.

**Example 6.1:** How to draw a decision tree to predict a student's academic performance based on the given information such as class attendance, class assignments, home-work assignments, tests, participation in competitions or other events, group activities such as projects and presentations, etc.

**Solution:** The target feature is the student performance in the final examination whether he will pass or fail in the examination. The decision nodes are test nodes which check for conditions like 'What's the student's class attendance?', 'How did he perform in his class assignments?', 'Did he do his home assignments properly?' 'What about his assessment results?', 'Did he participate in competitions or other events?', 'What is the performance rating in group activities such as projects and presentations?'. Table 6.1 shows the attributes and set of values for each attribute.

**Table 6.1:** Attributes and Associated Values

| Attributes                                          | Values               |
|-----------------------------------------------------|----------------------|
| Class attendance                                    | Good, Average, Poor  |
| Class assignments                                   | Good, Moderate, Poor |
| Home-work assignments                               | Yes, No              |
| Assessment                                          | Good, Moderate, Poor |
| Participation in competitions or other events       | Yes, No              |
| Group activities such as projects and presentations | Yes, No              |
| Exam Result                                         | Pass, Fail           |

The leaf nodes represent the outcomes, that is, either 'pass', or 'fail'.

A decision tree would be constructed by following a set of if-else conditions which may or may not include all the attributes, and decision nodes outcomes are two or more than two. Hence, the tree is not a binary tree.

**Note:** A decision tree is not always a binary tree. It is a tree which can have more than two branches.

**Example 6.2:** Predict a student's academic performance of whether he will pass or fail based on the given information such as 'Assessment' and 'Assignment'. The following Table 6.2 shows the independent variables, Assessment and Assignment, and the target variable Exam Result with their values. Draw a binary decision tree.

**Table 6.2:** Attributes and Associated Values

| Attributes  | Values             |
|-------------|--------------------|
| Assessment  | $\geq 50$ , $< 50$ |
| Assignment  | Yes, No            |
| Exam Result | Pass, Fail         |

**Solution:** Consider the root node is 'Assessment'. If a student's marks are  $\geq 50$ , the root node is branched to leaf node 'Pass' and if the assessment marks are  $< 50$ , it is branched to another decision node. If the decision node in next level of the tree is 'Assignment' and if a student has submitted his assignment, the node branches to 'Pass' and if not submitted, the node branches to 'Fail'. Figure 6.2 depicts this rule.

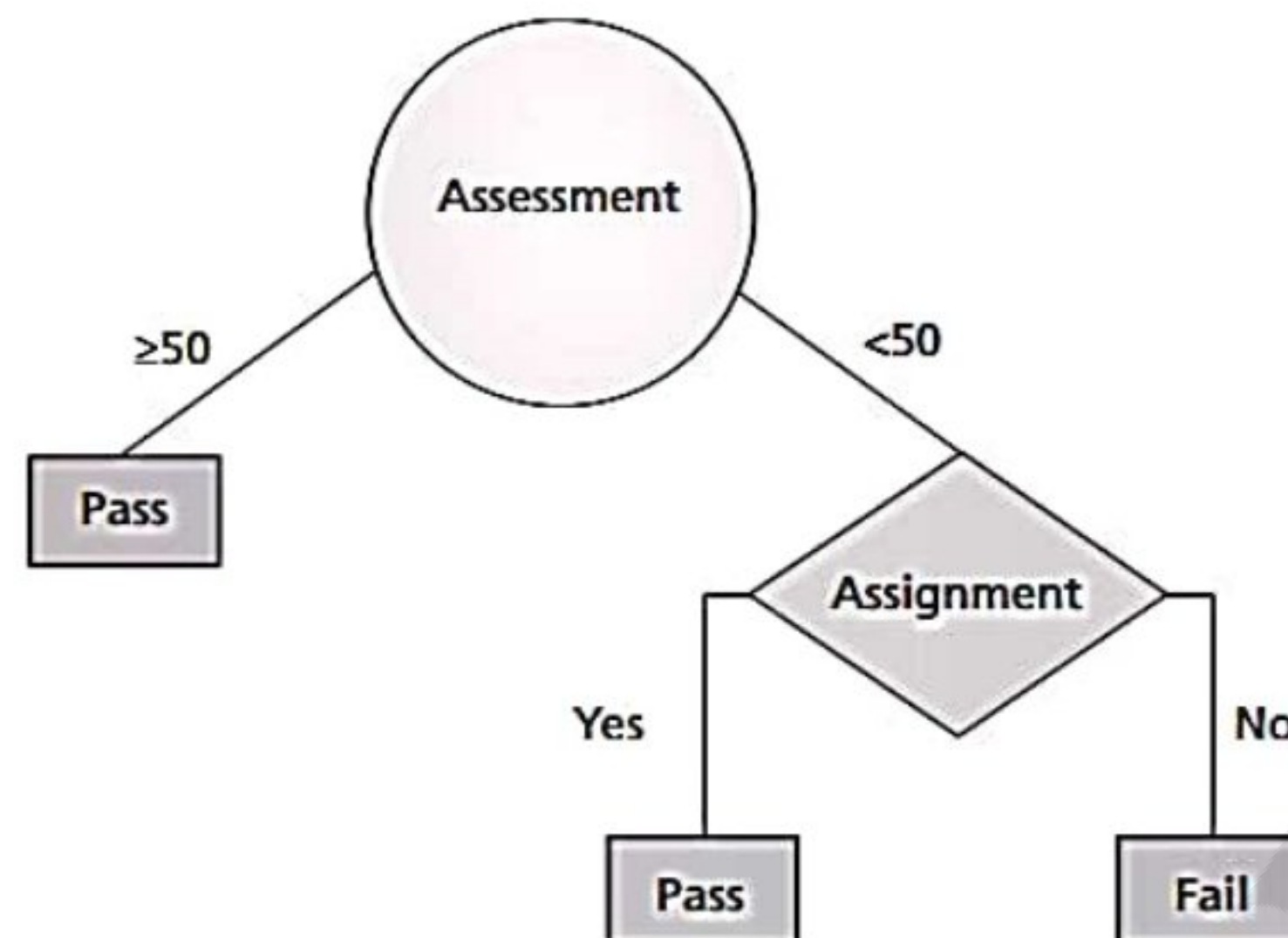


Figure 6.2: Illustration of a Decision Tree

This tree can be interpreted as a sequence of logical rules as follows:

if (Assessment  $\geq 50$ ) then 'Pass'

else if (Assessment  $< 50$ ) then

    if (Assignment = Yes) then 'Pass'

    else if (Assignment = No) then 'Fail'

Now, if a test instance is given, such as a student has scored 42 marks in his assessment and has not submitted his assignment, then it is predicted with the decision tree that his exam result is 'Fail'.

Many algorithms exist which will be studied for constructing decision trees in the sections below.

## 6.1.2 Fundamentals of Entropy

Given the training dataset with a set of attributes or features, the decision tree is constructed by finding the attribute or feature that best describes the target class for the given test instances. The best split feature is the one which contains more information about how to split the dataset among all features so that the target class is accurately identified for the test instances. In other words, the best split attribute is more informative to split the dataset into sub datasets and this process is continued until the stopping criterion is reached. This splitting should be pure at every stage of selecting the best feature.

The best feature is selected based on the amount of information among the features which are basically calculated on probabilities. Quantifying information is closely related to information theory. In the field of information theory, the features are quantified by a measure called Shannon Entropy which is calculated based on the probability distribution of the events.

Entropy is the amount of uncertainty or randomness in the outcome of a random variable or an event. Moreover, entropy describes about the homogeneity of the data instances. The best feature is selected based on the entropy value. For example, when a coin is flipped, head or tail are the two outcomes, hence its entropy is lower when compared to rolling a dice which has got six outcomes. Hence, the interpretation is,

Higher the entropy → Higher the uncertainty

Lower the entropy → Lower the uncertainty

Similarly, if all instances are homogenous, say (1, 0), which means all instances belong to the same class (here it is positive) or (0, 1) where all instances are negative, then the entropy is 0. On the other hand, if the instances are equally distributed, say (0.5, 0.5), which means 50% positive and 50% negative, then the entropy is 1. If there are 10 data instances, out of which 6 belong to positive class and 4 belong to negative class, then the entropy is calculated as shown in Eq. (6.1),

$$\text{Entropy} = - \left[ \frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10} \right] \quad (6.1)$$

It is concluded that if the dataset has instances that are completely homogeneous, then the entropy is 0 and if the dataset has samples that are equally divided (i.e., 50% – 50%), it has an entropy of 1. Thus, the entropy value ranges between 0 and 1 based on the randomness of the samples in the dataset. If the entropy is 0, then the split is pure which means that all samples in the set will partition into one class or category. But if the entropy is 1, the split is impure and the distribution of the samples is more random. The stopping criterion is based on the entropy value.

Let  $P$  be the probability distribution of data instances from 1 to  $n$  as shown in Eq. (6.2).

$$\text{So, } P = P_1, \dots, P_n \quad (6.2)$$

Entropy of  $P$  is the information measure of this probability distribution given in Eq. (6.3),

$$\begin{aligned} \text{Entropy\_Info}(P) &= \text{Entropy\_Info}(P_1, \dots, P_n) \\ &= -(P_1 \log_2(P_1) + P_2 \log_2(P_2) + \dots + P_n \log_2(P_n)) \end{aligned} \quad (6.3)$$

where,  $P_1$  is the probability of data instances classified as class 1 and  $P_2$  is the probability of data instances classified as class 2 and so on.

$P_1 = \text{No of data instances belonging to class 1} / \text{Total no of data instances in the training dataset}$

Entropy\_Info( $P$ ) can be computed as shown in Eq. (6.4).

Thus,

$$\text{Entropy\_Info}(6, 4) \text{ is calculated as } - \left[ \frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10} \right] \quad (6.4)$$

Mathematically, entropy is defined in Eq. (6.5) as:

$$\text{Entropy\_Info}(X) = \sum_{x \in \text{values}(X)} \text{Pr}[X = x] \cdot \log_2 \frac{1}{\text{Pr}[X = x]} \quad (6.5)$$

$\text{Pr}[X = x]$  is the probability of a random variable  $X$  with a possible outcome  $x$ .

$$\text{Note: } \log_2 \frac{1}{\text{Pr}[X = x]} = -\log_2(\text{Pr}[X = x])$$

### Algorithm 6.1: General Algorithm for Decision Trees

1. Find the best attribute from the training dataset using an attribute selection measure and place it at the root of the tree.

(Continued)

2. Split the training dataset into subsets based on the outcomes of the test attribute and each subset in a branch contains the data instances or tuples with the same value for the selected test attribute.
3. Repeat step 1 and step 2 on each subset until we end up in leaf nodes in all the branches of the tree.
4. This splitting process is recursive until the stopping criterion is reached.

### Stopping Criteria

The following are some of the common stopping conditions:

1. The data instances are homogenous which means all belong to the same class  $C_i$  and hence its entropy is 0.
2. A node with some defined minimum number of data instances becomes a leaf (Number of data instances in a node is between 0.25 and 1.00% of the full training dataset).
3. The maximum tree depth is reached, so further splitting is not done and the node becomes a leaf node.

## 6.2 DECISION TREE INDUCTION ALGORITHMS

There are many decision tree algorithms, such as ID3, C4.5, CART, CHAID, QUEST, GUIDE, CRUISE, and CTREE, that are used for classification in real-time environment. The most commonly used decision tree algorithms are ID3 (Iterative Dichotomizer 3), developed by J.R Quinlan in 1986, and C4.5 is an advancement of ID3 presented by the same author in 1993. CART, that stands for Classification and Regression Trees, is another algorithm which was developed by Breiman et al. in 1984.

The accuracy of the tree constructed depends upon the selection of the best split attribute. Different algorithms are used for building decision trees which use different measures to decide on the splitting criterion. Algorithms such as ID3, C4.5 and CART are popular algorithms used in the construction of decision trees. The algorithm ID3 uses 'Information Gain' as the splitting criterion whereas the algorithm C4.5 uses 'Gain Ratio' as the splitting criterion. The CART algorithm is popularly used for classifying both categorical and continuous-valued target variables. CART uses GINI Index to construct a decision tree.

Decision trees constructed using ID3 and C4.5 are also called as *univariate decision trees* which consider only one feature/attribute to split at each decision node whereas decision trees constructed using CART algorithm are *multivariate decision trees* which consider a conjunction of univariate splits. The details about univariate and multivariate data has been discussed in Chapter 2.

### 6.2.1 ID3 Tree Construction

ID3 is a supervised learning algorithm which uses a training dataset with labels and constructs a decision tree. ID3 is an example of univariate decision trees as it considers only one feature at each decision node. This leads to axis-aligned splits. The tree is then used to classify the future test instances. It constructs the tree using a greedy approach in a top-down fashion by identifying the best attribute at each level of the tree.

ID3 works well if the attributes or features are considered as discrete/categorical values. If some attributes are continuous, then we have to partition attributes or features to be discretized or nominal attributes or features.

The algorithm builds the tree using a purity measure called 'Information Gain' with the given training data instances and then uses the constructed tree to classify the test data. It is applied for training set with only nominal attributes or categorical attributes and with no missing values for classification. ID3 works well for a large dataset. If the dataset is small, overfitting may occur. Moreover, it is not accurate if the dataset has missing attribute values.

No pruning is done during or after construction of the tree and it is prone to outliers. C4.5 and CART can handle both categorical attributes and continuous attributes. Both C4.5 and CART can also handle missing values, but C4.5 is prone to outliers whereas CART can handle outliers as well.

#### Algorithm 6.2: Procedure to Construct a Decision Tree using ID3

1. Compute Entropy\_Info Eq. (6.8) for the whole training dataset based on the target attribute.
2. Compute Entropy\_Info Eq. (6.9) and Information\_Gain Eq. (6.10) for each of the attribute in the training dataset.
3. Choose the attribute for which entropy is minimum and therefore the gain is maximum as the best split attribute.
4. The best split attribute is placed as the root node.
5. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into subsets.
6. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

**Note:** We stop branching a node if entropy is 0.

*The best split attribute at every iteration is the attribute with the highest information gain.*

### Definitions

Let  $T$  be the training dataset.

Let  $A$  be the set of attributes  $A = \{A_1, A_2, A_3, \dots, A_n\}$ .

Let  $m$  be the number of classes in the training dataset.

Let  $P_i$  be the probability that a data instance or a tuple ' $d$ ' belongs to class  $C_i$ .

It is calculated as,

$P_i = \text{Total no of data instances that belongs to class } C_i \text{ in } T / \text{Total no of tuples in the training set } T$  (6.6)

Mathematically, it is represented as shown in Eq. (6.7).

$$P_i = \frac{|d_{C_i}|}{|T|} \quad (6.7)$$

Expected information or Entropy needed to classify a data instance  $d'$  in  $T$  is denoted as  $\text{Entropy\_Info}(T)$  given in Eq. (6.8).

$$\text{Entropy\_Info}(T) = - \sum_{i=1}^m P_i \log_2 P_i \quad (6.8)$$

Entropy of every attribute denoted as  $\text{Entropy\_Info}(T, A)$  is shown in Eq. (6.9) as:

$$\text{Entropy\_Info}(T, A) = \sum_{i=1}^v \frac{|A_i|}{|T|} \times \text{Entropy\_Info}(A_i) \quad (6.9)$$

where, the attribute  $A$  has got ' $v$ ' distinct values  $\{a_1, a_2, \dots, a_v\}$ ,  $|A_i|$  is the number of instances for distinct value ' $i$ ' in attribute  $A$ , and  $\text{Entropy\_Info}(A_i)$  is the entropy for that set of instances.

Information\_Gain is a metric that measures how much information is gained by branching on an attribute  $A$ . In other words, it measures the reduction in impurity in an arbitrary subset of data.

It is calculated as given in Eq. (6.10):

$$\text{Information\_Gain}(A) = \text{Entropy\_Info}(T) - \text{Entropy\_Info}(T, A) \quad (6.10)$$

It can be noted that as entropy increases, information gain decreases. They are inversely proportional to each other.

Scan for 'Additional Examples'



**Example 6.3:** Assess a student's performance during his course of study and predict whether a student will get a job offer or not in his final year of the course. The training dataset  $T$  consists of 10 data instances with attributes such as 'CGPA', 'Interactiveness', 'Practical Knowledge' and 'Communication Skills' as shown in Table 6.3. The target class attribute is the 'Job Offer'.

Table 6.3: Training Dataset  $T$

| S.No. | CGPA     | Interactiveness | Practical Knowledge | Communication Skills | Job Offer |
|-------|----------|-----------------|---------------------|----------------------|-----------|
| 1.    | $\geq 9$ | Yes             | Very good           | Good                 | Yes       |
| 2.    | $\geq 8$ | No              | Good                | Moderate             | Yes       |
| 3.    | $\geq 9$ | No              | Average             | Poor                 | No        |
| 4.    | $< 8$    | No              | Average             | Good                 | No        |
| 5.    | $\geq 8$ | Yes             | Good                | Moderate             | Yes       |
| 6.    | $\geq 9$ | Yes             | Good                | Moderate             | Yes       |
| 7.    | $< 8$    | Yes             | Good                | Poor                 | No        |
| 8.    | $\geq 9$ | No              | Very good           | Good                 | Yes       |
| 9.    | $\geq 8$ | Yes             | Good                | Good                 | Yes       |
| 10.   | $\geq 8$ | Yes             | Average             | Good                 | Yes       |

**Solution:**

**Step 1:**

Calculate the Entropy for the target class 'Job Offer'.

$$\begin{aligned} \text{Entropy\_Info}(\text{Target Attribute} = \text{Job Offer}) &= \text{Entropy\_Info}(7, 3) = \\ &= -\left[ \frac{7}{10} \log_2 \frac{7}{10} + \frac{3}{10} \log_2 \frac{3}{10} \right] = -(-0.3599 + -0.5208) = 0.8807 \end{aligned}$$

**Iteration 1:**

**Step 2:**

Calculate the Entropy\_Info and Gain(Information\_Gain) for each of the attribute in the training dataset.

Table 6.4 shows the number of data instances classified with Job Offer as Yes or No for the attribute CGPA.

**Table 6.4: Entropy Information for CGPA**

| CGPA     | Job Offer = Yes | Job Offer = No | Total | Entropy |
|----------|-----------------|----------------|-------|---------|
| $\geq 9$ | 3               | 1              | 4     |         |
| $\geq 8$ | 4               | 0              | 4     | 0       |
| $< 8$    | 0               | 2              | 2     | 0       |

Entropy\_Info(T, CGPA)

$$\begin{aligned} &= \frac{4}{10} \left[ -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \right] + \frac{4}{10} \left[ -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} \right] + \frac{2}{10} \left[ -\frac{0}{2} \log_2 \frac{0}{2} - \frac{2}{2} \log_2 \frac{2}{2} \right] \\ &= \frac{4}{10} (0.3111 + 0.4997) + 0 + 0 \\ &= 0.3243 \end{aligned}$$

$$\begin{aligned} \text{Gain (CGPA)} &= 0.8807 - 0.3243 \\ &= 0.5564 \end{aligned}$$

Table 6.5 shows the number of data instances classified with Job Offer as Yes or No for the attribute Interactiveness.

**Table 6.5: Entropy Information for Interactiveness**

| Interactiveness | Job Offer = Yes | Job Offer = No | Total | Entropy |
|-----------------|-----------------|----------------|-------|---------|
| YES             | 5               | 1              | 6     |         |
| NO              | 2               | 2              | 4     |         |

$$\begin{aligned} \text{Entropy\_Info}(T, \text{Interactiveness}) &= \frac{6}{10} \left[ -\frac{5}{6} \log_2 \frac{5}{6} - \frac{1}{6} \log_2 \frac{1}{6} \right] + \frac{4}{10} \left[ -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} \right] \\ &= \frac{6}{10} (0.2191 + 0.4306) + \frac{4}{10} (0.4997 + 0.4997) \\ &= 0.3898 + 0.3998 = 0.7896 \end{aligned}$$

$$\begin{aligned} \text{Gain(Interactiveness)} &= 0.8807 - 0.7896 \\ &= 0.0911 \end{aligned}$$

Table 6.6 shows the number of data instances classified with Job Offer as Yes or No for the attribute Practical Knowledge.

Table 6.6: Entropy Information for Practical Knowledge

| Practical Knowledge | Job Offer = Yes | Job Offer = No | Total | Entropy |
|---------------------|-----------------|----------------|-------|---------|
| Very Good           | 2               | 0              | 2     | 0       |
| Average             | 1               | 2              | 3     |         |
| Good                | 4               | 1              | 5     |         |

Entropy\_Info( $T$ , Practical Knowledge)

$$\begin{aligned}
 &= \frac{2}{10} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{3}{10} \left[ -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \right] + \frac{5}{10} \left[ -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right] \\
 &= \frac{2}{10}(0) + \frac{3}{10}(0.5280 + 0.3897) + \frac{5}{10}(0.2574 + 0.4641) \\
 &= 0 + 0.2753 + 0.3608 \\
 &= 0.6361
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain(Practical Knowledge)} &= 0.8807 - 0.6361 \\
 &= 0.2446
 \end{aligned}$$

Table 6.7 shows the number of data instances classified with Job Offer as Yes or No for attribute Communication Skills.

Table 6.7: Entropy Information for Communication Skills

| Communication Skills | Job Offer = Yes | Job Offer = No | Total |
|----------------------|-----------------|----------------|-------|
| Good                 | 4               | 1              | 5     |
| Moderate             | 3               | 0              | 3     |
| Poor                 | 0               | 2              | 2     |

Entropy\_Info( $T$ , Communication Skills)

$$\begin{aligned}
 &= \frac{5}{10} \left[ -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right] + \frac{3}{10} \left[ -\frac{3}{3} \log_2 \frac{3}{3} - \frac{0}{3} \log_2 \frac{0}{3} \right] + \frac{2}{10} \left[ -\frac{0}{2} \log_2 \frac{0}{2} - \frac{2}{2} \log_2 \frac{2}{2} \right] \\
 &= \frac{5}{10}(0.5280 + 0.3897) + \frac{3}{10}(0) + \frac{2}{10}(0) \\
 &= 0.3609
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain(Communication Skills)} &= 0.8813 - 0.36096 \\
 &= 0.5203
 \end{aligned}$$

The Gain calculated for all the attributes is shown in Table 6.8:

Table 6.8: Gain

| Attributes           | Gain   |
|----------------------|--------|
| CGPA                 | 0.5564 |
| Interactiveness      | 0.0911 |
| Practical Knowledge  | 0.2246 |
| Communication Skills | 0.5203 |

**Step 3:** From Table 6.8, choose the attribute for which entropy is minimum and therefore the gain is maximum as the best split attribute.

The best split attribute is CGPA since it has the maximum gain. So, we choose CGPA as the root node. There are three distinct values for CGPA with outcomes  $\geq 9$ ,  $\geq 8$  and  $< 8$ . The entropy value is 0 for  $\geq 8$  and  $< 8$  with all instances classified as Job Offer = Yes for  $\geq 8$  and Job Offer = No for  $< 8$ . Hence, both  $\geq 8$  and  $< 8$  end up in a leaf node. The tree grows with the subset of instances with CGPA  $\geq 9$  as shown in Figure 6.3.

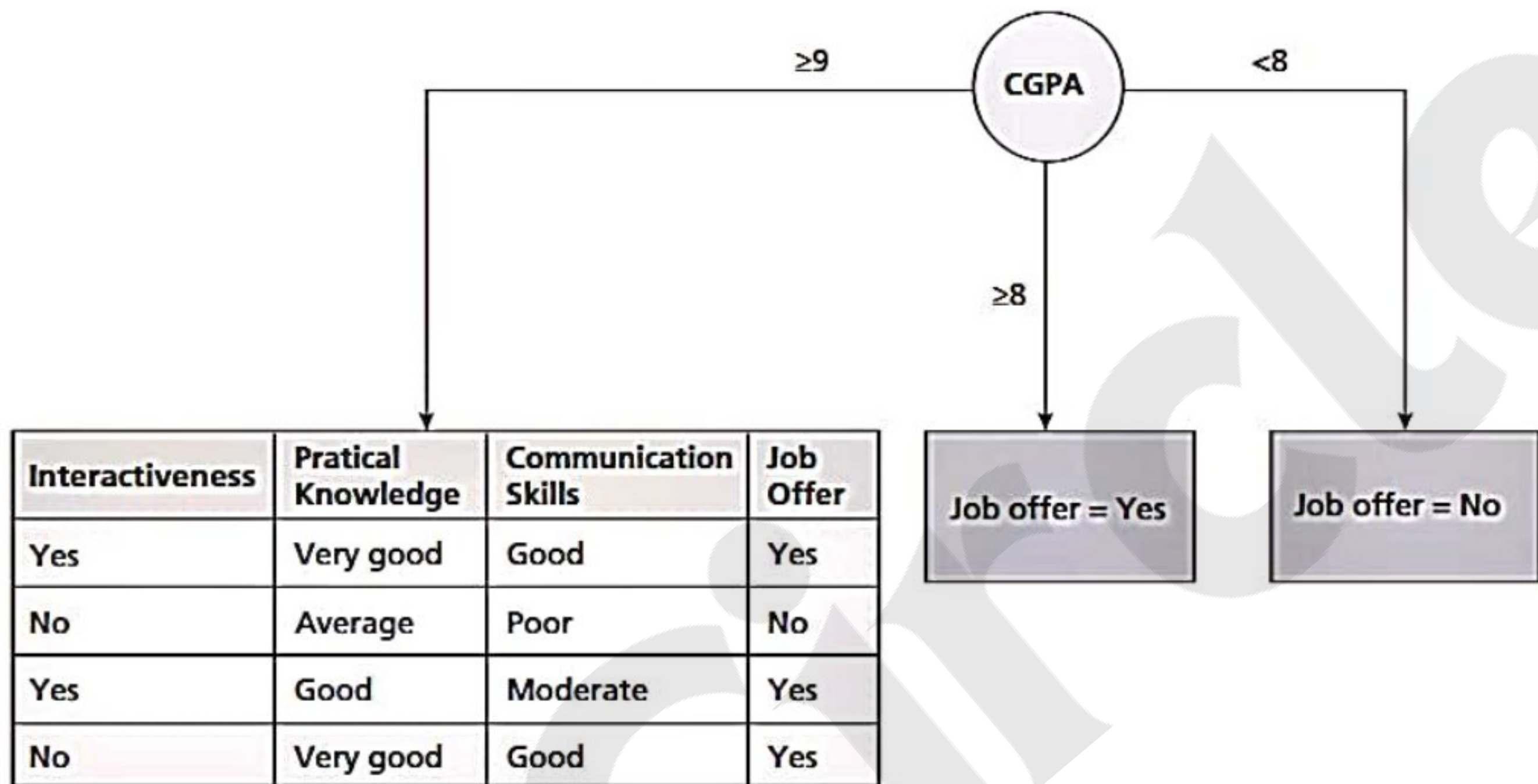


Figure 6.3: Decision Tree After Iteration 1

Now, continue the same process for the subset of data instances branched with CGPA  $\geq 9$ .

#### Iteration 2:

In this iteration, the same process of computing the Entropy\_Info and Gain are repeated with the subset of training set. The subset consists of 4 data instances as shown in the above Figure 6.3.

$$\text{Entropy\_Info}(T) = \text{Entropy\_Info}(3, 1) =$$

$$= -\left[ \frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right]$$

$$= -(-0.3111 + -0.4997)$$

$$= 0.8108$$

$$\text{Entropy\_Info}(T, \text{Interactiveness}) = \frac{2}{4} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{2}{4} \left[ -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} \right]$$

$$= 0 + 0.4997$$

$$\text{Gain}(\text{Interactiveness}) = 0.8108 - 0.4997$$

$$= 0.3111$$

$$\text{Entropy\_Info}(T, \text{Practical Knowledge})$$

$$= \frac{2}{4} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{1}{4} \left[ -\frac{0}{1} \log_2 \frac{0}{1} - \frac{1}{1} \log_2 \frac{1}{1} \right] + \frac{1}{4} \left[ -\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1} \right]$$

$$= 0$$

$$\text{Gain}(\text{Practical Knowledge}) = 0.8108$$

$$\text{Entropy\_Info}(T, \text{Communication Skills})$$

$$= \frac{2}{4} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{1}{4} \left[ -\frac{0}{1} \log_2 \frac{0}{1} - \frac{1}{1} \log_2 \frac{1}{1} \right] + \frac{1}{4} \left[ -\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1} \right]$$

$$= 0$$

$$\text{Gain}(\text{Communication Skills}) = 0.8108$$

The gain calculated for all the attributes is shown in Table 6.9.

Table 6.9: Total Gain

| Attributes           | Gain   |
|----------------------|--------|
| Interactiveness      | 0.3111 |
| Practical Knowledge  | 0.8108 |
| Communication Skills | 0.8108 |

Here, both the attributes 'Practical Knowledge' and 'Communication Skills' have the same Gain. So, we can either construct the decision tree using 'Practical Knowledge' or 'Communication Skills'. The final decision tree is shown in Figure 6.4.

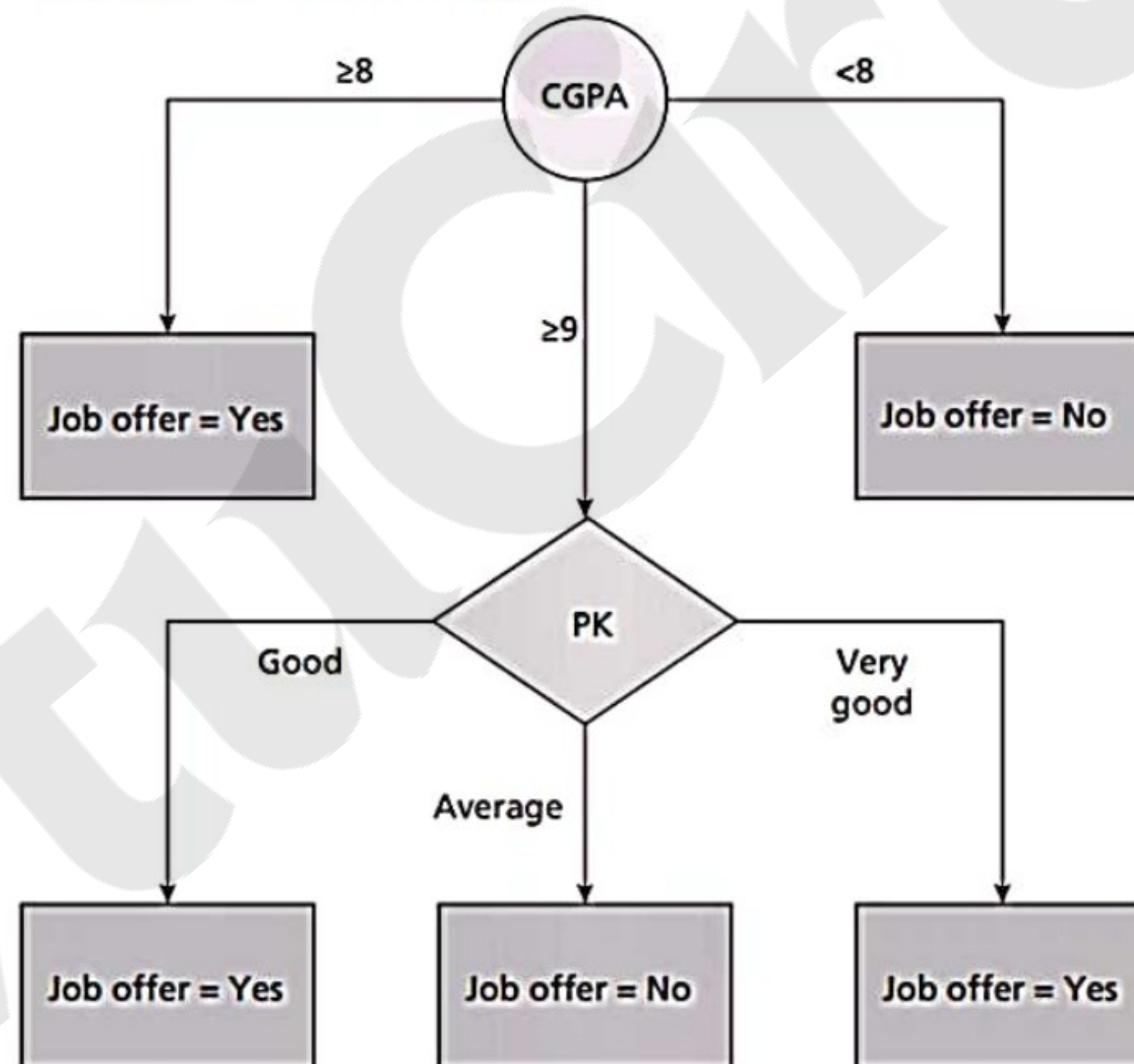


Figure 6.4: Final Decision Tree

## 6.2.2 C4.5 Construction

C4.5 is an improvement over ID3. C4.5 works with continuous and discrete attributes and missing values, and it also supports post-pruning. C5.0 is the successor of C4.5 and is more efficient and used for building smaller decision trees. C4.5 works with missing values by marking as '?', but these missing attribute values are not considered in the calculations.

The algorithm C4.5 is based on Occam's Razor which says that given two correct solutions, the simpler solution has to be chosen. Moreover, the algorithm requires a larger training set for better accuracy. It uses Gain Ratio as a measure during the construction of decision trees. ID3 is more biased towards attributes with larger values. For example, if there is an attribute called 'Register No' for students it would be unique for every student and will have distinct value for every data instance resulting in more values for the attribute. Hence, every instance belongs to a category and would have higher Information Gain than other attributes. To overcome this bias issue, C4.5 uses a purity measure Gain ratio to identify the best split attribute. In C4.5 algorithm, the Information Gain measure used in ID3 algorithm is normalized by computing another factor called Split\_Info. This normalized information gain of an attribute called as Gain\_Ratio is computed by the ratio of the calculated Split\_Info and Information Gain of each attribute. Then, the attribute with the highest normalized information gain, that is, highest gain ratio is used as the splitting criteria.

As an example, we will choose the same training dataset shown in Table 6.3 to construct a decision tree using the C4.5 algorithm.

Given a Training dataset  $T$ ,

The Split\_Info of an attribute  $A$  is computed as given in Eq. (6.11):

$$\text{Split\_Info}(T, A) = -\sum_{i=1}^v \frac{|A_i|}{|T|} \times \log_2 \frac{|A_i|}{|T|} \quad (6.11)$$

where, the attribute  $A$  has got ' $v$ ' distinct values  $\{a_1, a_2, \dots, a_v\}$ , and  $|A_i|$  is the number of instances for distinct value ' $i$ ' in attribute  $A$ .

The Gain\_Ratio of an attribute  $A$  is computed as given in Eq. (6.12):

$$\text{Gain\_Ratio}(A) = \frac{\text{Info\_Gain}(A)}{\text{Split\_Info}(T, A)} \quad (6.12)$$

#### Algorithm 6.3: Procedure to Construct a Decision Tree using C4.5

1. Compute Entropy\_Info Eq. (6.8) for the whole training dataset based on the target attribute.
2. Compute Entropy\_Info Eq. (6.9), Info\_Gain Eq. (6.10), Split\_Info Eq. (6.11) and Gain\_Ratio Eq. (6.12) for each of the attribute in the training dataset.
3. Choose the attribute for which Gain\_Ratio is maximum as the best split attribute.
4. The best split attribute is placed as the root node.
5. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into subsets.
6. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

**Example 6.4:** Make use of Information Gain of the attributes which are calculated in ID3 algorithm in Example 6.3 to construct a decision tree using C4.5.

**Solution:**

**Iteration 1:**

**Step 1:** Calculate the Class\_Entropy for the target class 'Job Offer'.

$$\begin{aligned}\text{Entropy\_Info}(\text{Target Attribute} = \text{Job Offer}) &= \text{Entropy\_Info}(7, 3) = \\ &= -\left[\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right] \\ &= (-0.3599 + -0.5208) \\ &= 0.8807\end{aligned}$$

**Step 2:** Calculate the Entropy\_Info, Gain(Info\_Gain), Split\_Info, Gain\_Ratio for each of the attribute in the training dataset.

**CGPA:**

$$\begin{aligned}\text{Entropy Info}(T, \text{CGPA}) &= \frac{4}{10}\left[-\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4}\right] + \frac{4}{10}\left[-\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4}\right] \\ &\quad + \frac{2}{10}\left[-\frac{0}{2}\log_2\frac{0}{2} - \frac{2}{2}\log_2\frac{2}{2}\right] \\ &= \frac{4}{10}(0.3111 + 0.4997) + 0 + 0 \\ &= 0.3243\end{aligned}$$

$$\begin{aligned}\text{Gain}(\text{CGPA}) &= 0.8807 - 0.3243 \\ &= 0.5564\end{aligned}$$

$$\begin{aligned}\text{Split\_Info}(T, \text{CGPA}) &= -\frac{4}{10}\log_2\frac{4}{10} - \frac{4}{10}\log_2\frac{4}{10} - \frac{2}{10}\log_2\frac{2}{10} \\ &= 0.5285 + 0.5285 + 0.4641 \\ &= 1.5211\end{aligned}$$

$$\begin{aligned}\text{Gain Ratio}(\text{CGPA}) &= (\text{Gain}(\text{CGPA})) / (\text{Split\_Info}(T, \text{CGPA})) \\ &= \frac{0.5564}{1.5211} = 0.3658\end{aligned}$$

**Interactiveness:**

$$\begin{aligned}\text{Entropy Info}(T, \text{Interactiveness}) &= \frac{6}{10}\left[-\frac{5}{6}\log_2\frac{5}{6} - \frac{1}{6}\log_2\frac{1}{6}\right] + \frac{4}{10}\left[-\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4}\right] \\ &= \frac{6}{10}(0.2191 + 0.4306) + \frac{4}{10}(0.4997 + 0.4997) \\ &= 0.3898 + 0.3998 = 0.7896\end{aligned}$$

$$\text{Gain}(\text{Interactiveness}) = 0.8807 - 0.7896 = 0.0911$$

$$\text{Gain}(\text{Interactiveness}) = -\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10} = 0.9704$$

$$\begin{aligned}
 \text{Gain\_Ratio(Interactiveness)} &= \frac{\text{Gain(Interactiveness)}}{\text{Split\_Info}(T, \text{Interactiveness})} \\
 &= \frac{0.0911}{0.9704} \\
 &= 0.0939
 \end{aligned}$$

**Practical Knowledge:**

$$\begin{aligned}
 \text{Entropy\_Info}(T, \text{Practical Knowledge}) &= \frac{2}{10} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{3}{10} \left[ -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \right] \\
 &\quad + \frac{5}{10} \left[ -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right] \\
 &= \frac{2}{10}(0) + \frac{3}{10}(0.5280 + 0.3897) + \frac{5}{10}(0.2574 + 0.4641) \\
 &= 0 + 0.2753 + 0.3608 = 0.6361
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain(Practical Knowledge)} &= 0.8807 - 0.6361 \\
 &= 0.2448
 \end{aligned}$$

$$\begin{aligned}
 \text{Split\_Info}(T, \text{Practical Knowledge}) &= -\frac{2}{10} \log_2 \frac{2}{10} - \frac{5}{10} \log_2 \frac{5}{10} - \frac{3}{10} \log_2 \frac{3}{10} \\
 &= 1.4853
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain\_Ratio(Practical Knowledge)} &= \frac{\text{Gain(Practical Knowledge)}}{\text{Split\_Info}(T, \text{Practical Knowledge})} \\
 &= \frac{0.2448}{1.4853} \\
 &= 0.1648
 \end{aligned}$$

**Communication Skills:**

$$\begin{aligned}
 \text{Entropy\_Info}(T, \text{Communication Skills}) &= \frac{5}{10} \left[ -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \right] + \frac{3}{10} \left[ -\frac{3}{3} \log_2 \frac{3}{3} - \frac{0}{3} \log_2 \frac{0}{3} \right] \\
 &\quad + \frac{2}{10} \left[ -\frac{0}{2} \log_2 \frac{0}{2} - \frac{2}{2} \log_2 \frac{2}{2} \right] \\
 &= \frac{5}{10}(0.5280 + 0.3897) + \frac{3}{10}(0) + \frac{2}{10}(0) \\
 &= 0.3609
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain(Communication Skills)} &= 0.8813 - 0.36096 \\
 &= 0.5202
 \end{aligned}$$

$$\begin{aligned}
 \text{Split\_Info}(T, \text{Communication Skills}) &= -\frac{5}{10} \log_2 \frac{5}{10} - \frac{3}{10} \log_2 \frac{3}{10} - \frac{2}{10} \log_2 \frac{2}{10} \\
 &= 1.4853
 \end{aligned}$$

$$\begin{aligned}
 \text{Gain\_Ratio(Communication Skills)} &= \frac{\text{Gain(Communication Skills)}}{\text{Split\_Info}(T, \text{Communication Skills})} \\
 &= \frac{0.5202}{1.4853} = 0.3502
 \end{aligned}$$

Table 6.10 shows the Gain\_Ratio computed for all the attributes.

Table 6.10: Gain\_Ratio

| Attribute            | Gain_Ratio |
|----------------------|------------|
| CGPA                 | 0.3658     |
| INTERACTIVENESS      | 0.0939     |
| PRACTICAL KNOWLEDGE  | 0.1648     |
| COMMUNICATION SKILLS | 0.3502     |

**Step 3:** Choose the attribute for which Gain\_Ratio is maximum as the best split attribute.

From Table 6.10, we can see that CGPA has highest gain ratio and it is selected as the best split attribute. We can construct the decision tree placing CGPA as the root node shown in Figure 6.5. The training dataset is split into subsets with 4 data instances.

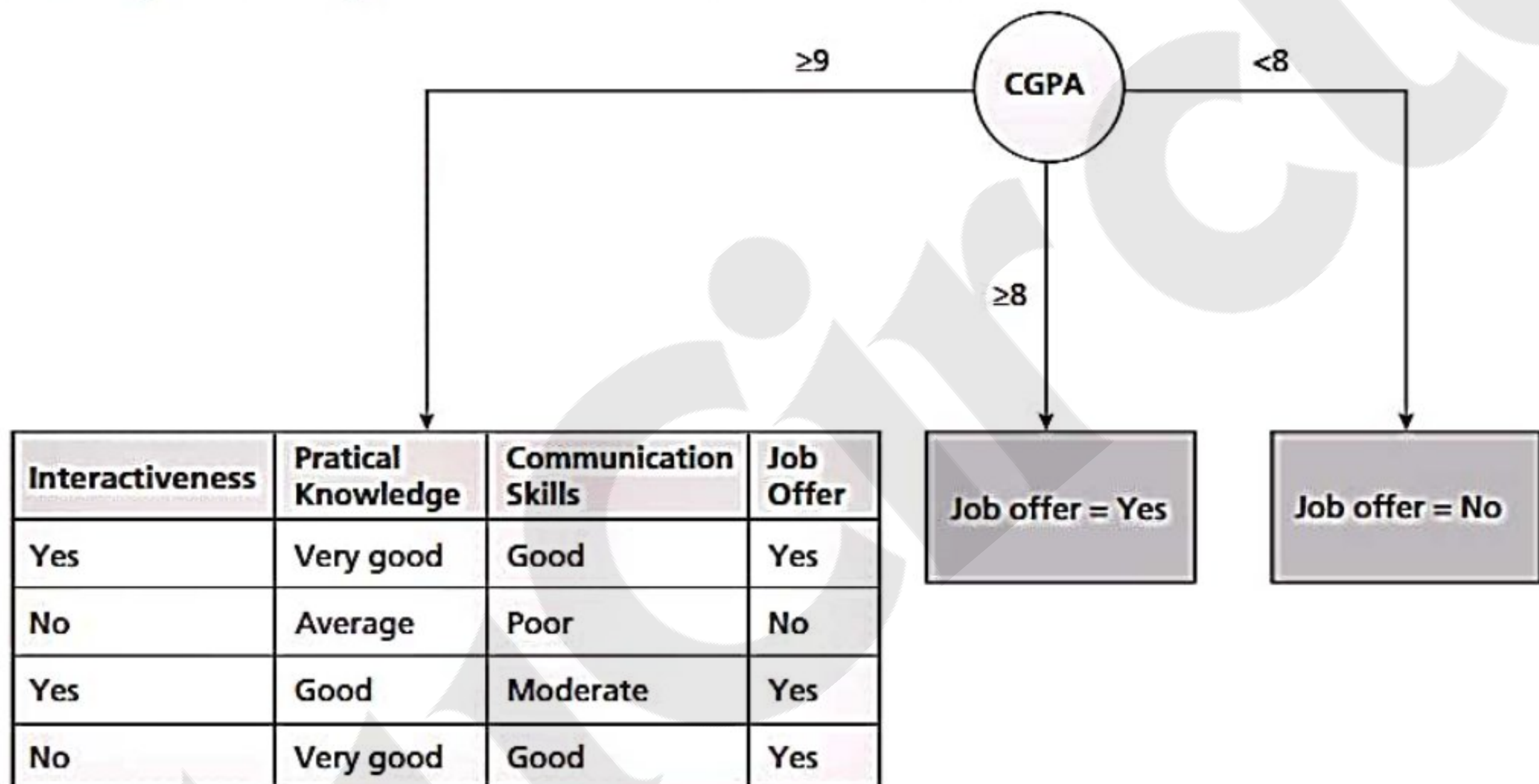


Figure 6.5: Decision Tree after Iteration 1

**Iteration 2:**

**Total Samples: 4**

Repeat the same process for this resultant dataset with 4 data instances.

Job Offer has 3 instances as Yes and 1 instance as No.

$$\begin{aligned}
 \text{Entropy\_Info}(\text{Target Class} = \text{Job Offer}) &= -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \\
 &= 0.3112 + 0.5 \\
 &= 0.8112
 \end{aligned}$$

**Interactiveness:**

$$\begin{aligned}
 \text{Entropy\_Info}(T, \text{Interactiveness}) &= \frac{2}{4} \left[ -\frac{2}{2} \log_2 \frac{2}{2} - \frac{0}{2} \log_2 \frac{0}{2} \right] + \frac{2}{4} \left[ -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} \right] \\
 &= 0 + 0.4997
 \end{aligned}$$

$$\text{Gain}(\text{Interactiveness}) = 0.8108 - 0.4997 = 0.3111$$

$$\text{Split\_Info}(T, \text{Interactiveness}) = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{2}{4}\log_2 \frac{2}{4} = 0.5 + 0.5 = 1$$

$$\begin{aligned}\text{Gain\_Ratio}(\text{Interactiveness}) &= \frac{\text{Gain}(\text{Interactiveness})}{\text{Split\_Info}(T, \text{Interactiveness})} \\ &= \frac{0.3112}{1} = 0.3112\end{aligned}$$

**Practical Knowledge:**

$$\begin{aligned}\text{Entropy\_Info}(T, \text{Practical Knowledge}) &= \frac{2}{4}\left[-\frac{2}{2}\log_2 \frac{2}{2} - \frac{0}{2}\log_2 \frac{0}{2}\right] + \frac{1}{4}\left[-\frac{0}{1}\log_2 \frac{0}{1} - \frac{1}{1}\log_2 \frac{1}{1}\right] \\ &\quad + \frac{1}{4}\left[-\frac{1}{1}\log_2 \frac{1}{1} - \frac{0}{1}\log_2 \frac{0}{1}\right] \\ &= 0\end{aligned}$$

$$\text{Gain}(\text{Practical Knowledge}) = 0.8108$$

$$\text{Split\_Info}(T, \text{Practical Knowledge}) = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{1}{4}\log_2 \frac{1}{4} - \frac{1}{4}\log_2 \frac{1}{4} = 1.5$$

$$\text{Gain\_Ratio}(\text{Practical Knowledge}) = \frac{\text{Gain}(\text{Practical Knowledge})}{\text{Split\_Info}(T, \text{Practical Knowledge})} = \frac{0.8108}{1.5} = 0.5408$$

**Communication Skills:**

$$\begin{aligned}\text{Entropy\_Info}(T, \text{Communication Skills}) &= \frac{2}{4}\left[-\frac{2}{2}\log_2 \frac{2}{2} - \frac{0}{2}\log_2 \frac{0}{2}\right] + \frac{1}{4}\left[-\frac{0}{1}\log_2 \frac{0}{1} - \frac{1}{1}\log_2 \frac{1}{1}\right] \\ &\quad + \frac{1}{4}\left[-\frac{1}{1}\log_2 \frac{1}{1} - \frac{0}{1}\log_2 \frac{0}{1}\right] \\ &= 0\end{aligned}$$

$$\text{Gain}(\text{Communication Skills}) = 0.8108$$

$$\text{Split\_Info}(T, \text{Communication Skills}) = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{1}{4}\log_2 \frac{1}{4} - \frac{1}{4}\log_2 \frac{1}{4} = 1.5$$

$$\text{Gain\_Ratio}(\text{Communication Skills}) = \frac{\text{Gain}(\text{Practical Knowledge})}{\text{Split\_Info}(T, \text{Practical Knowledge})} = \frac{0.8108}{1.5} = 0.5408$$

Table 6.11 shows the Gain\_Ratio computed for all the attributes.

**Table 6.11: Gain-Ratio**

| Attributes           | Gain_Ratio |
|----------------------|------------|
| Interactiveness      | 0.3112     |
| Practical Knowledge  | 0.5408     |
| Communication Skills | 0.5408     |

Both 'Practical Knowledge' and 'Communication Skills' have the highest gain ratio. So, the best splitting attribute can either be 'Practical Knowledge' or 'Communication Skills', and therefore, the split can be based on any one of these.

Here, we split based on 'Practical Knowledge'. The final decision tree is shown in Figure 6.6.

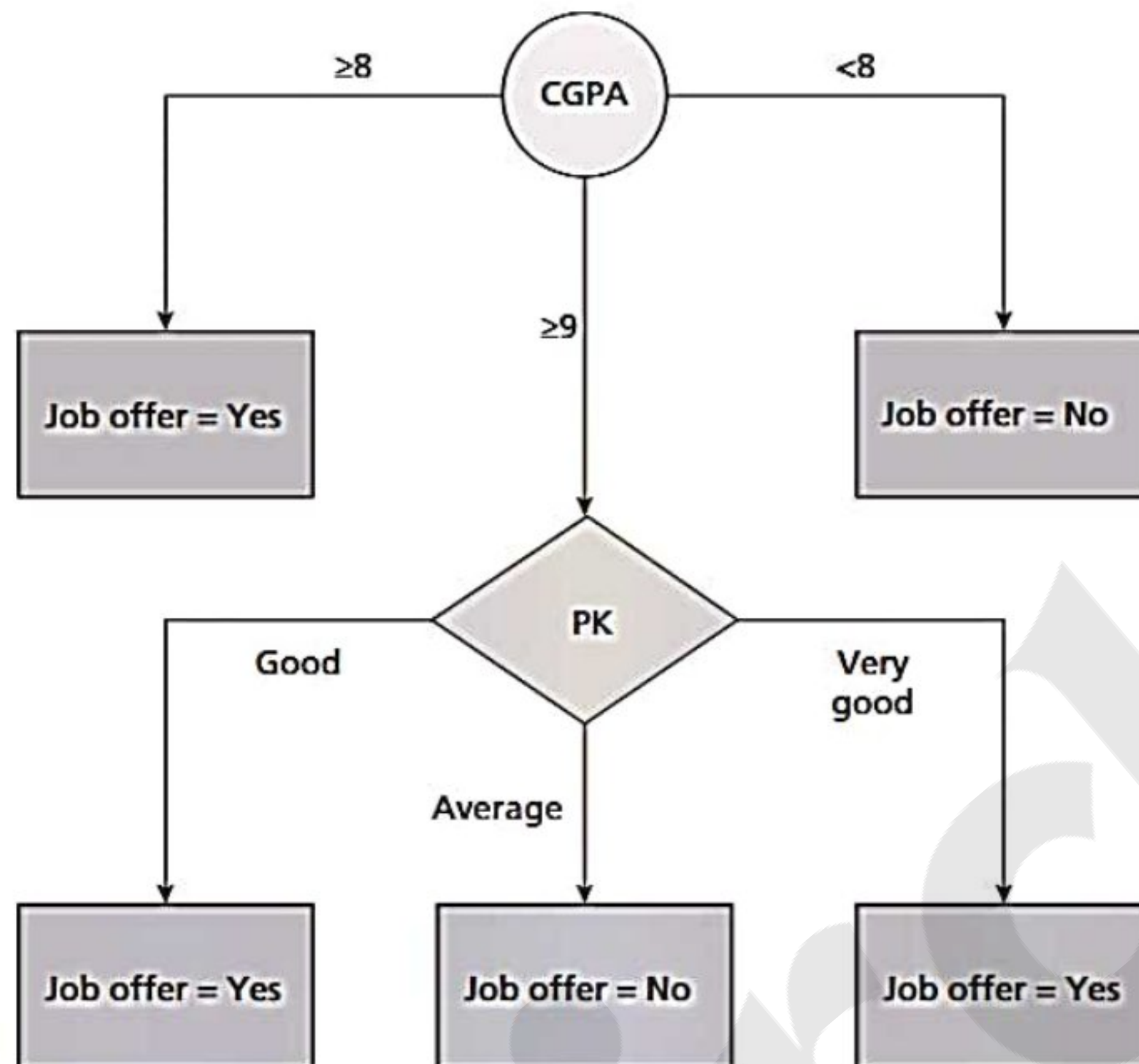


Figure 6.6: Final Decision Tree

### Dealing with Continuous Attributes in C4.5

The C4.5 algorithm is further improved by considering attributes which are continuous, and a continuous attribute is discretized by finding a split point or threshold. When an attribute 'A' has numerical values which are continuous, a threshold or best split point 's' is found such that the set of values is categorized into two sets such as  $A < s$  and  $A \geq s$ . The best split point is the attribute value which has maximum information gain for that attribute.

Now, let us consider the set of continuous values for the attribute CGPA in the sample dataset as shown in Table 6.12.

Table 6.12: Sample Dataset

| S.No. | CGPA | Job Offer |
|-------|------|-----------|
| 1.    | 9.5  | Yes       |
| 2.    | 8.2  | Yes       |
| 3.    | 9.1  | No        |
| 4.    | 6.8  | No        |
| 5.    | 8.5  | Yes       |
| 6.    | 9.5  | Yes       |
| 7.    | 7.9  | No        |
| 8.    | 9.1  | Yes       |
| 9.    | 8.8  | Yes       |
| 10.   | 8.8  | Yes       |

First, sort the values in an ascending order.

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 6.8 | 7.9 | 8.2 | 8.5 | 8.8 | 8.8 | 9.1 | 9.1 | 9.5 | 9.5 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

Remove the duplicates and consider only the unique values of the attribute.

|     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|
| 6.8 | 7.9 | 8.2 | 8.5 | 8.8 | 9.1 | 9.5 |
|-----|-----|-----|-----|-----|-----|-----|

Now, compute the Gain for the distinct values of this continuous attribute. Table 6.13 shows the computed values.

Table 6.13: Gain Values for CGPA

|                     | 6.8    |        | 7.9    |        | 8.2    |        | 8.5    |        | 8.8    |        | 9.1    |   | 9.5    |   |
|---------------------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|---|--------|---|
| Range               | ≤      | >      | ≤      | >      | ≤      | >      | ≤      | >      | ≤      | >      | ≤      | > | ≤      | > |
| Yes                 | 0      | 7      | 0      | 7      | 1      | 6      | 2      | 5      | 4      | 3      | 5      | 2 | 7      | 0 |
| No                  | 1      | 2      | 2      | 1      | 2      | 1      | 2      | 1      | 2      | 1      | 3      | 0 | 3      | 0 |
| Entropy             | 0      | 0.7637 | 0      | 0.5433 | 0.9177 | 0.5913 | 1      | 0.6497 | 0.9177 | 0.8108 | 0.9538 | 0 | 0.8808 | 0 |
| Entropy_Info (S, T) | 0.6873 |        | 0.4346 |        | 0.6892 |        | 0.7898 |        | 0.8749 |        | 0.7630 |   | 0.8808 |   |
| Gain                | 0.1935 |        | 0.4462 |        | 0.1916 |        | 0.091  |        | 0.0059 |        | 0.1178 |   | 0      |   |

For a sample, the calculations are shown below for a single distinct value say,  $CGPA \in 6.8$ .

$$\begin{aligned} \text{Entropy\_Info}(T, \text{Job\_Offer}) &= -\left[ \frac{7}{10} \log_2 \frac{7}{10} + \frac{3}{10} \log_2 \frac{3}{10} \right] \\ &= -(-0.3599 + -0.5209) \\ &= 0.8808 \end{aligned}$$

$$\begin{aligned} \text{Entropy}(7, 2) &= -\left[ \frac{7}{9} \log_2 \frac{7}{9} + \frac{2}{9} \log_2 \frac{2}{9} \right] \\ &= -(-0.2818 + -0.4819) \\ &= 0.7637 \end{aligned}$$

$$\begin{aligned} \text{Entropy\_Info}(T, CGPA \in 6.8) &= \frac{1}{10} \times \text{Entropy}(0, 1) + \frac{9}{10} \text{Entropy}(7, 2) \\ &= \frac{1}{10} \left[ -\frac{0}{1} \log_2 \frac{0}{1} - \frac{1}{1} \log_2 \frac{1}{1} \right] + \frac{9}{10} \left[ -\frac{7}{9} \log_2 \frac{7}{9} - \frac{2}{9} \log_2 \frac{2}{9} \right] \\ &= 0 + \frac{9}{10} (0.7637) \\ &= 0.6873 \end{aligned}$$

$$\begin{aligned} \text{Gain}(CGPA \in 6.8) &= 0.8808 - 0.6873 \\ &= 0.1935 \end{aligned}$$

Similarly, the calculations are done for each of the distinct value for the attribute CGPA and a table is created. Now, the value of CGPA with maximum gain is chosen as the threshold value or the best split point. From Table 6.13, we can observe that CGPA with 7.9 has the maximum gain as 0.4462. Hence,  $CGPA \in 7.9$  is chosen as the split point. Now, we can discretize the continuous values of CGPA as two categories with  $CGPA \leq 7.9$  and  $CGPA > 7.9$ . The resulting discretized instances are shown in Table 6.14.

Table 6.14: Discretized Instances

| S.No. | CGPA Continuous | CGPA Discretized | Job Offer |
|-------|-----------------|------------------|-----------|
| 1.    | 9.5             | >7.9             | Yes       |
| 2.    | 8.2             | >7.9             | Yes       |
| 3.    | 9.1             | >7.9             | No        |
| 4.    | 6.8             | ≤7.9             | No        |
| 5.    | 8.5             | >7.9             | Yes       |
| 6.    | 9.5             | >7.9             | Yes       |
| 7.    | 7.9             | ≤7.9             | No        |
| 8.    | 9.1             | >7.9             | Yes       |
| 9.    | 8.8             | >7.9             | Yes       |
| 10.   | 8.8             | >7.9             | Yes       |

### 6.2.3 Classification and Regression Trees Construction

The Classification and Regression Trees (CART) algorithm is a multivariate decision tree learning used for classifying both categorical and continuous-valued target variables. CART algorithm is an example of multivariate decision trees that gives oblique splits. It solves both classification and regression problems. If the target feature is categorical, it constructs a classification tree and if the target feature is continuous, it constructs a regression tree. CART uses GINI Index to construct a decision tree. GINI Index is defined as the number of data instances for a class or it is the proportion of instances. It constructs the tree as a binary tree by recursively splitting a node into two nodes. Therefore, even if an attribute has more than two possible values, GINI Index is calculated for all subsets of the attributes and the subset which has maximum value is selected as the best split subset. For example, if an attribute  $A$  has three distinct values say  $\{a_1, a_2, a_3\}$ , the possible subsets are  $\{\}, \{a_1\}, \{a_2\}, \{a_3\}, \{a_1, a_2\}, \{a_1, a_3\}, \{a_2, a_3\}$ , and  $\{a_1, a_2, a_3\}$ . So, if an attribute has 3 distinct values, the number of possible subsets is  $2^3$ , which means 8. Excluding the empty set  $\{\}$  and the full set  $\{a_1, a_2, a_3\}$ , we have 6 subsets. With 6 subsets, we can form three possible combinations such as:

$\{a_1\}$  with  $\{a_2, a_3\}$

$\{a_2\}$  with  $\{a_1, a_3\}$

$\{a_3\}$  with  $\{a_1, a_2\}$

Hence, in this CART algorithm, we need to compute the best splitting attribute and the best split subset  $i$  in the chosen attribute.

Higher the GINI value, higher is the homogeneity of the data instances.

Gini\_Index( $T$ ) is computed as given in Eq. (6.13).

$$\text{Gini\_Index}(T) = 1 - \sum_{i=1}^m P_i^2 \quad (6.13)$$

where,

$P_i$  be the probability that a data instance or a tuple ' $d$ ' belongs to class  $C_i$ . It is computed as:

$P_i = \text{No. of data instances belonging to class } i / \text{Total no of data instances in the training dataset } T$

GINI Index assumes a binary split on each attribute, therefore, every attribute is considered as a binary attribute which splits the data instances into two subsets  $S_1$  and  $S_2$ .

$\text{Gini\_Index}(T, A)$  is computed as given in Eq. (6.14).

$$\text{Gini\_Index}(T, A) = \frac{|S_1|}{|T|} \text{Gini}(S_1) + \frac{|S_2|}{|T|} \text{Gini}(S_2) \quad (6.14)$$

The splitting subset with minimum  $\text{Gini\_Index}$  is chosen as the best splitting subset for an attribute. The best splitting attribute is chosen by the minimum  $\text{Gini\_Index}$  which is otherwise maximum  $\Delta\text{Gini}$  because it reduces the impurity.

$\Delta\text{Gini}$  is computed as given in Eq. (6.15):

$$\Delta\text{Gini}(A) = \text{Gini}(T) - \text{Gini}(T, A) \quad (6.15)$$

#### Algorithm 6.4: Procedure to Construct a Decision Tree using CART

1. Compute  $\text{Gini\_Index}$  Eq. (6.13) for the whole training dataset based on the target attribute.
2. Compute  $\text{Gini\_Index}$  for each of the attribute Eq. (6.14) and for the subsets of each attribute in the training dataset.
3. Choose the best splitting subset which has minimum  $\text{Gini\_Index}$  for an attribute.
4. Compute  $\Delta\text{Gini}$  Eq. (6.15) for the best splitting subset of that attribute.
5. Choose the best splitting attribute that has maximum  $\Delta\text{Gini}$ .
6. The best split attribute with the best split subset is placed as the root node.
7. The root node is branched into two subtrees with each subtree an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into two subsets.
8. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

**Example 6.5:** Choose the same training dataset shown in Table 6.3 and construct a decision tree using CART algorithm.

**Solution:**

**Step 1:** Calculate the  $\text{Gini\_Index}$  for the dataset shown in Table 6.3, which consists of 10 data instances. The target attribute 'Job Offer' has 7 instances as Yes and 3 instances as No.

$$\begin{aligned} \text{Gini\_Index}(T) &= 1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2 \\ &= 1 - 0.49 - 0.09 \\ &= 1 - 0.58 \end{aligned}$$

$$\text{Gini\_Index}(T) = 0.42$$

**Step 2:** Compute  $\text{Gini\_Index}$  for each of the attribute and each of the subset in the attribute.

CGPA has 3 categories, so there are 6 subsets and hence 3 combinations of subsets (as shown in Table 6.15).

Table 6.15: Categories of CGPA

| CGPA     | Job Offer = Yes | Job Offer = No |
|----------|-----------------|----------------|
| $\geq 9$ | 3               | 1              |
| $\geq 8$ | 4               | 0              |
| $< 8$    | 0               | 2              |

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{\geq 9, \geq 8\}) &= 1 - (7/8)^2 - (1/8)^2 \\ &= 1 - 0.7806 \\ &= 0.2194 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{< 8\}) &= 1 - (0/2)^2 - (2/2)^2 \\ &= 1 - 1 \\ &= 0 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{(\geq 9, \geq 8), < 8\}) &= (8/10) \times 0.2194 + (2/10) \times 0 \\ &= 0.17552 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{\geq 9, < 8\}) &= 1 - (3/6)^2 - (3/6)^2 \\ &= 1 - 0.5 = 0.5 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{\geq 8\}) &= 1 - (4/4)^2 - (0/4)^2 \\ &= 1 - 1 = 0 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{(\geq 9, < 8), \geq 8\}) &= (6/10) \times 0.5 + (4/10) \times 0 \\ &= 0.3 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{\geq 8, < 8\}) &= 1 - (4/6)^2 - (2/6)^2 \\ &= 1 - 0.555 \\ &= 0.445 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{\geq 9\}) &= 1 - (3/4)^2 - (1/4)^2 \\ &= 1 - 0.625 \\ &= 0.375 \end{aligned}$$

$$\begin{aligned} \text{Gini\_Index}(T, \text{CGPA} \in \{(\geq 8, < 8), \geq 9\}) &= (6/10) \times 0.445 + (4/10) \times 0.375 \\ &= 0.417 \end{aligned}$$

Table 6.16 shows the Gini\_Index for 3 subsets of CGPA.

Table 6.16: Gini\_Index of CGPA

| Subsets            |          | Gini_Index |
|--------------------|----------|------------|
| $(\geq 9, \geq 8)$ | $< 8$    | 0.1755     |
| $(\geq 9, < 8)$    | $\geq 8$ | 0.3        |
| $(\geq 8, < 8)$    | $\geq 9$ | 0.417      |

**Step 3:** Choose the best splitting subset which has minimum Gini\_Index for an attribute.

The subset  $\text{CGPA} \in \{(\geq 9, \geq 8), < 8\}$  has the lowest Gini\_Index value as 0.1755 is chosen as the best splitting subset.

**Step 4:** Compute  $\Delta\text{Gini}$  or the best splitting subset of that attribute.

$$\begin{aligned}\Delta\text{Gini}(\text{CGPA}) &= \text{Gini}(T) - \text{Gini}(T, \text{CGPA}) \\ &= 0.42 - 0.1755 \\ &= 0.2445\end{aligned}$$

Repeat the same process for the remaining attributes in the dataset such as for Interactiveness shown in Table 6.17, Practical Knowledge in Table 6.18, and Communication Skills in Table 6.20.

**Table 6.17:** Categories for Interactiveness

| Interactiveness | Job Offer = Yes | Job Offer = No |
|-----------------|-----------------|----------------|
| Yes             | 5               | 1              |
| No              | 2               | 2              |

$$\begin{aligned}\text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{Yes}\}) &= 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 \\ &= 1 - 0.72 \\ &= 0.28\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{No}\}) &= 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 \\ &= 1 - 0.5 \\ &= 0.5\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{Yes}, \text{No}\}) &= \frac{6}{10}(0.28) + \frac{4}{10}(0.5) \\ &= 0.168 + 0.2 \\ &= 0.368\end{aligned}$$

$$\begin{aligned}\Delta\text{Gini}(\text{Interactiveness}) &= \text{Gini}(T) - \text{Gini}(T, \text{Interactiveness}) \\ &= 0.42 - 0.368 \\ &= 0.052\end{aligned}$$

**Table 6.18:** Categories for Practical Knowledge

| Practical Knowledge | Job Offer = Yes | Job Offer = No |
|---------------------|-----------------|----------------|
| Very Good           | 2               | 0              |
| Good                | 4               | 1              |
| Average             | 1               | 2              |

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good}, \text{Good}\}) &= \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 \\ &= 1 - 0.7544 \\ &= 0.2456\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Average}\}) &= 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 \\ &= 1 - 0.555 = 0.445\end{aligned}$$

$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Good}\}, \text{Average})$

$$= \left(\frac{7}{10}\right)^2 \times 0.2456 + \left(\frac{3}{10}\right)^2 \times 0.445$$

$$= 0.3054$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Average}\}) = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2$$

$$= 1 - 0.52$$

$$= 0.48$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Good}\}) = 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2$$

$$= 1 - 0.68$$

$$= 0.32$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Average}\}, \text{Good}) = \left(\frac{5}{10}\right) \times 0.48 + \left(\frac{5}{10}\right) \times 0.32$$

$$= 0.40$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Average}\}) = 1 - \left(\frac{5}{8}\right)^2 - \left(\frac{3}{8}\right)^2$$

$$= 1 - 0.5312 = 0.4688$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good}\}) = 1 - \left(\frac{2}{2}\right)^2 - \left(\frac{0}{2}\right)^2$$

$$= 1 - 1 = 0$$

$$\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Good, Average}\}, \text{Very Good}) = \left(\frac{8}{10}\right) \times 0.4688 + \left(\frac{2}{10}\right) \times 0$$

$$= 0.3750$$

Table 6.19 shows the Gini\_Index for various subsets of Practical Knowledge.

**Table 6.19: Gini\_Index for Practical Knowledge**

| Subsets              |           | Gini_Index |
|----------------------|-----------|------------|
| (Very Good, Good)    | Average   | 0.3054     |
| (Very Good, Average) | Good      | 0.40       |
| (Good, Average)      | Very Good | 0.3750     |

$$\Delta\text{Gini}(\text{Practical Knowledge}) = \text{Gini}(T) - \text{Gini}(T, \text{Practical Knowledge})$$

$$= 0.42 - 0.3054 = 0.1146$$

**Table 6.20: Categories for Communication Skills**

| Communication Skills | Job Offer = Yes | Job Offer = No |
|----------------------|-----------------|----------------|
| Good                 | 4               | 1              |
| Moderate             | 3               | 0              |
| Poor                 | 0               | 2              |

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Moderate}\}) &= 1 - \left(\frac{7}{8}\right)^2 - \left(\frac{1}{8}\right)^2 \\ &= 1 - 0.7806 \\ &= 0.2194\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Poor}\}) &= 1 - \left(\frac{2}{2}\right)^2 - \left(\frac{0}{2}\right)^2 \\ &= 1 - 1 = 0\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Moderate}, \text{Poor}\}) &= \left(\frac{8}{10}\right) \times 0.2194 + \left(\frac{2}{10}\right) \times 0 \\ &= 0.1755\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Poor}\}) &= 1 - \left(\frac{4}{7}\right)^2 - \left(\frac{3}{7}\right)^2 \\ &= 1 - 0.5101 \\ &= 0.4899\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate}\}) &= 1 - \left(\frac{3}{3}\right)^2 - \left(\frac{0}{3}\right)^2 \\ &= 1 - 1 = 0\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Poor}, \text{Moderate}\}) &= \left(\frac{7}{10}\right) \times 0.4899 + \left(\frac{3}{10}\right) \times 0 \\ &= 0.3429\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate, Poor}\}) &= 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 \\ &= 1 - 0.52 \\ &= 0.48\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good}\}) &= 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 \\ &= 1 - 0.68 \\ &= 0.32\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate, Poor}, \text{Good}\}) &= \left(\frac{5}{10}\right)^2 \times 0.48 + \left(\frac{5}{10}\right)^2 \times 0.32 \\ &= 0.40\end{aligned}$$

Table 6.21 shows the Gini\_Index for various subsets of Communication Skills.

**Table 6.21: Gini-Index for Subsets of Communication Skills**

| Subsets          |          | Gini_Index |
|------------------|----------|------------|
| (Good, Moderate) | Poor     | 0.1755     |
| (Good, Poor)     | Moderate | 0.3429     |
| (Moderate, Poor) | Good     | 0.40       |

$$\begin{aligned}
 \Delta\text{Gini}(\text{Communication Skills}) &= \text{Gini}(T) - \text{Gini}(T, \text{Communication Skills}) \\
 &= 0.42 - 0.1755 \\
 &= 0.2445
 \end{aligned}$$

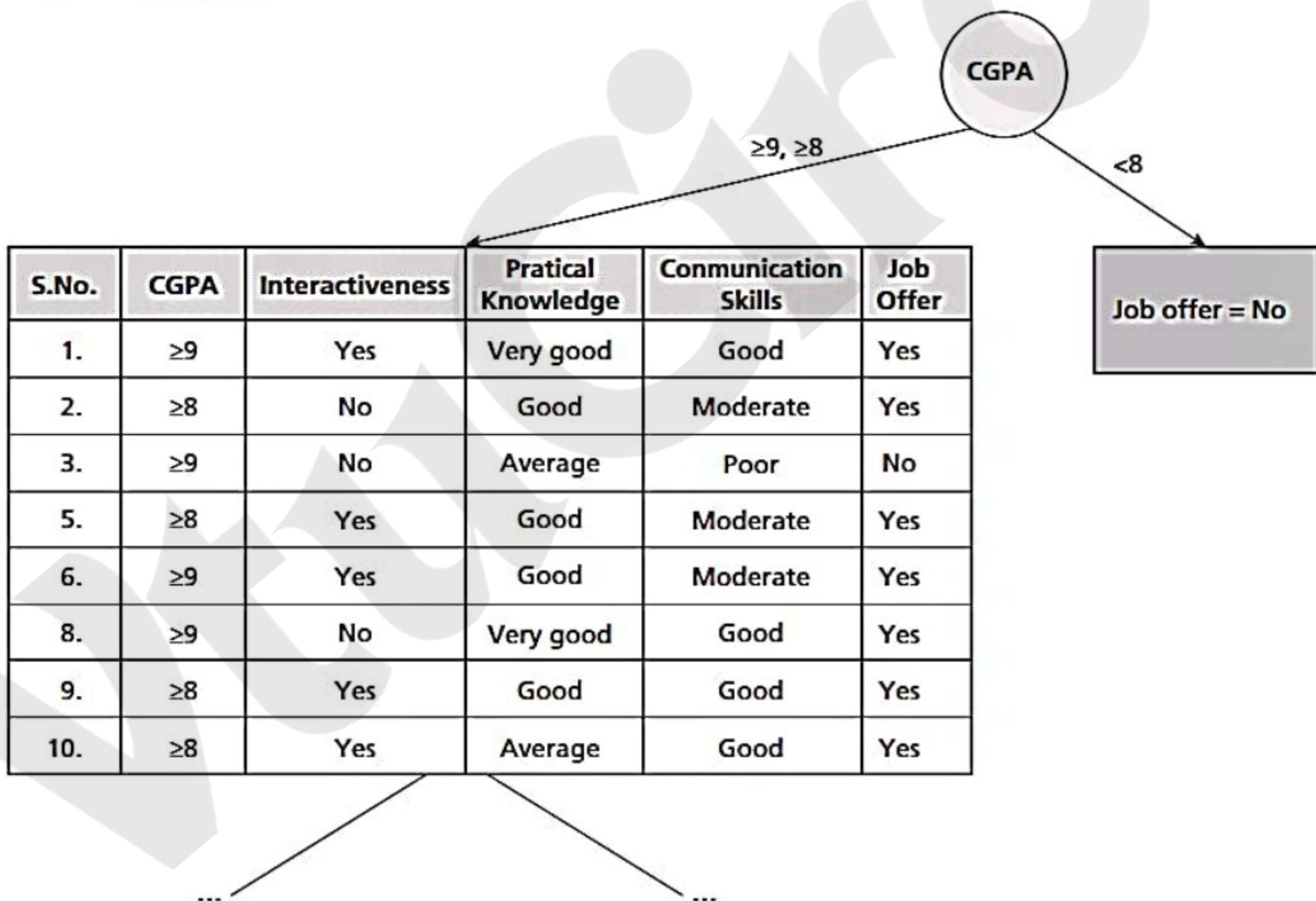
Table 6.22 shows the Gini\_Index and  $\Delta\text{Gini}$  values calculated for all the attributes.

**Table 6.22:** Gini\_Index and  $\Delta\text{Gini}$  for all Attributes

| Attribute            | Gini_Index | $\Delta\text{Gini}$ |
|----------------------|------------|---------------------|
| CGPA                 | 0.1755     | 0.2445              |
| Interactiveness      | 0.368      | 0.052               |
| Practical knowledge  | 0.3054     | 0.1146              |
| Communication Skills | 0.1755     | 0.2445              |

**Step 5:** Choose the best splitting attribute that has maximum  $\Delta\text{Gini}$ .

CGPA and Communication Skills have the highest  $\Delta\text{Gini}$  value. We can choose CGPA as the root node and split the datasets into two subsets shown in Figure 6.7 since the tree constructed by CART is a binary tree.



**Figure 6.7:** Decision Tree after Iteration 1

### Iteration 2:

In the second iteration, the dataset has 8 data instances as shown in Table 6.23. Repeat the same process to find the best splitting attribute and the splitting subset for that attribute.

Table 6.23: Subset of the Training Dataset after Iteration 1

| S.No. | CGPA     | Interactiveness | Practical Knowledge | Communication Skills | Job Offer |
|-------|----------|-----------------|---------------------|----------------------|-----------|
| 1.    | $\geq 9$ | Yes             | Very good           | Good                 | Yes       |
| 2.    | $\geq 8$ | No              | Good                | Moderate             | Yes       |
| 3.    | $\geq 9$ | No              | Average             | Poor                 | No        |
| 5.    | $\geq 8$ | Yes             | Good                | Moderate             | Yes       |
| 6.    | $\geq 9$ | Yes             | Good                | Moderate             | Yes       |
| 8.    | $\geq 9$ | No              | Very good           | Good                 | Yes       |
| 9.    | $\geq 8$ | Yes             | Good                | Good                 | Yes       |
| 10.   | $\geq 8$ | Yes             | Average             | Good                 | Yes       |

$$\begin{aligned}
 \text{Gini\_Index}(T) &= 1 - \left(\frac{7}{8}\right)^2 - \left(\frac{1}{8}\right)^2 \\
 &= 1 - 0.766 - 0.0156 \\
 &= 1 - 0.58
 \end{aligned}$$

$$\text{Gini\_Index}(T) = 0.2184$$

Tables 6.24, 6.25, and 6.27 show the categories for attributes Interactiveness, Practical Knowledge, and Communication Skills, respectively.

Table 6.24: Categories for Interactiveness

| Interactiveness | Job Offer = Yes | Job Offer = No |
|-----------------|-----------------|----------------|
| Yes             | 5               | 0              |
| No              | 2               | 1              |

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{Yes}\}) &= 1 - \left(\frac{5}{5}\right)^2 - \left(\frac{0}{5}\right)^2 \\
 &= 1 - 1 = 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{No}\}) &= 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 \\
 &= 1 - 0.44 - 0.111 = 0.449
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Interactiveness} \in \{\text{Yes}, \text{No}\}) &= \left(\frac{7}{8}\right) \times 0 + \left(\frac{1}{8}\right) \times 0.449 \\
 &= 0.056
 \end{aligned}$$

$$\begin{aligned}
 \Delta\text{Gini}(\text{Interactiveness}) &= \text{Gini}(T) - \text{Gini}(T, \text{Interactiveness}) \\
 &= 0.2184 - 0.056 = 0.1624
 \end{aligned}$$

Table 6.25: Categories for Practical Knowledge

| Practical Knowledge | Job Offer = Yes | Job Offer = No |
|---------------------|-----------------|----------------|
| Very Good           | 2               | 0              |
| Good                | 4               | 0              |
| Average             | 1               | 1              |

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Good}\}) &= 1 - \left(\frac{6}{6}\right)^2 - \left(\frac{0}{6}\right)^2 \\ &= 1 - 1 = 0\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Average}\}) &= 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 \\ &= 1 - 0.25 - 0.25 \\ &= 0.5\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Good}, \text{Average}\}) &= \left(\frac{6}{8}\right)^2 \times 0 + \left(\frac{2}{8}\right)^2 \times 0.5 \\ &= 0.125\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Average}\}) &= 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 \\ &= 1 - 0.5625 - 0.0625 \\ &= 0.375\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Good}\}) &= 1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 \\ &= 1 - 1 = 0\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good, Average}, \text{Good}\}) &= \left(\frac{4}{8}\right)^2 \times 0.375 + \left(\frac{4}{8}\right)^2 \times 0 \\ &= 0.1875\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Good, Average}\}) &= 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 \\ &= 1 - 0.694 - 0.028 \\ &= 0.278\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Very Good}\}) &= 1 - \left(\frac{2}{2}\right)^2 - \left(\frac{0}{2}\right)^2 \\ &= 1 - 1 = 0\end{aligned}$$

$$\begin{aligned}\text{Gini\_Index}(T, \text{Practical Knowledge} \in \{\text{Good, Average}, \text{Very Good}\}) &= \left(\frac{6}{8}\right)^2 \times 0.278 + \left(\frac{2}{8}\right)^2 \times 0 \\ &= 0.2085\end{aligned}$$

Table 6.26 shows the Gini\_Index values for various subsets of Practical Knowledge.

**Table 6.26: Gini\_Index for Subsets of Practical Knowledge**

| Subsets              |           | Gini_Index |
|----------------------|-----------|------------|
| (Very Good, Good)    | Average   | 0.125      |
| (Very Good, Average) | Good      | 0.1875     |
| (Good, Average)      | Very Good | 0.2085     |

$$\begin{aligned}
 \Delta \text{Gini}(\text{Practical Knowledge}) &= \text{Gini}(T) - \text{Gini}(T, \text{Practical Knowledge}) \\
 &= 0.2184 - 0.125 \\
 &= 0.0934
 \end{aligned}$$

Table 6.27: Categories for Communication Skills

| Communication Skills | Job Offer = Yes | Job Offer = No |
|----------------------|-----------------|----------------|
| Good                 | 4               | 0              |
| Moderate             | 3               | 0              |
| Poor                 | 0               | 1              |

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Moderate}\}) &= 1 - \left(\frac{7}{7}\right)^2 - \left(\frac{0}{7}\right)^2 \\
 &= 1 - 1 = 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Poor}\}) &= 1 - \left(\frac{0}{1}\right)^2 - \left(\frac{1}{1}\right)^2 \\
 &= 1 - 1 = 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Moderate, Poor}\}) &= \left(\frac{7}{8}\right)^2 \times 0 + \left(\frac{1}{8}\right)^2 \times 0 \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Poor}\}) &= 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 \\
 &= 1 - 0.64 - 0.04 \\
 &= 0.32
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate}\}) &= 1 - \left(\frac{3}{3}\right)^2 - \left(\frac{0}{3}\right)^2 \\
 &= 1 - 1 = 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good, Poor, Moderate}\}) &= \left(\frac{5}{8}\right)^2 \times 0.32 + \left(\frac{3}{8}\right)^2 \times 0 \\
 &= 0.2
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate, Poor}\}) &= 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 \\
 &= 1 - 0.5625 - 0.0625 \\
 &= 0.375
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Good}\}) &= 1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 \\
 &= 1 - 1 = 0
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini\_Index}(T, \text{Communication Skills} \in \{\text{Moderate, Poor, Good}\}) &= \left(\frac{4}{8}\right)^2 \times 0.375 + \left(\frac{4}{8}\right)^2 \times 0 \\
 &= 0.1875
 \end{aligned}$$

Table 6.28 shows the Gini\_Index for subsets of Communication Skills.

Table 6.28: Gini\_Index for Subsets of Communication Skills

| Subsets          |          | Gini_Index |
|------------------|----------|------------|
| (Good, Moderate) | Poor     | 0          |
| (Good, Poor)     | Moderate | 0.2        |
| (Moderate, Poor) | Good     | 0.1875     |

$$\begin{aligned}\Delta\text{Gini}(\text{Communication Skills}) &= \text{Gini}(T) - \text{Gini}(T, \text{Communication Skills}) \\ &= 0.2184 - 0 = 0.2184\end{aligned}$$

Table 6.29 shows the Gini\_Index and  $\Delta\text{Gini}$  values for all attributes.

Table 6.29: Gini\_Index and  $\Delta\text{Gini}$  Values for All Attributes

| Attribute            | Gini_Index | $\Delta\text{Gini}$ |
|----------------------|------------|---------------------|
| Interactiveness      | 0.056      | 0.1624              |
| Practical knowledge  | 0.125      | 0.0934              |
| Communication Skills | 0          | 0.2184              |

Communication Skills has the highest  $\Delta\text{Gini}$  value. The tree is further branched based on the attribute 'Communication Skills'. Here, we see all branches end up in a leaf node and the process of construction is completed. The final tree is shown in Figure 6.8.

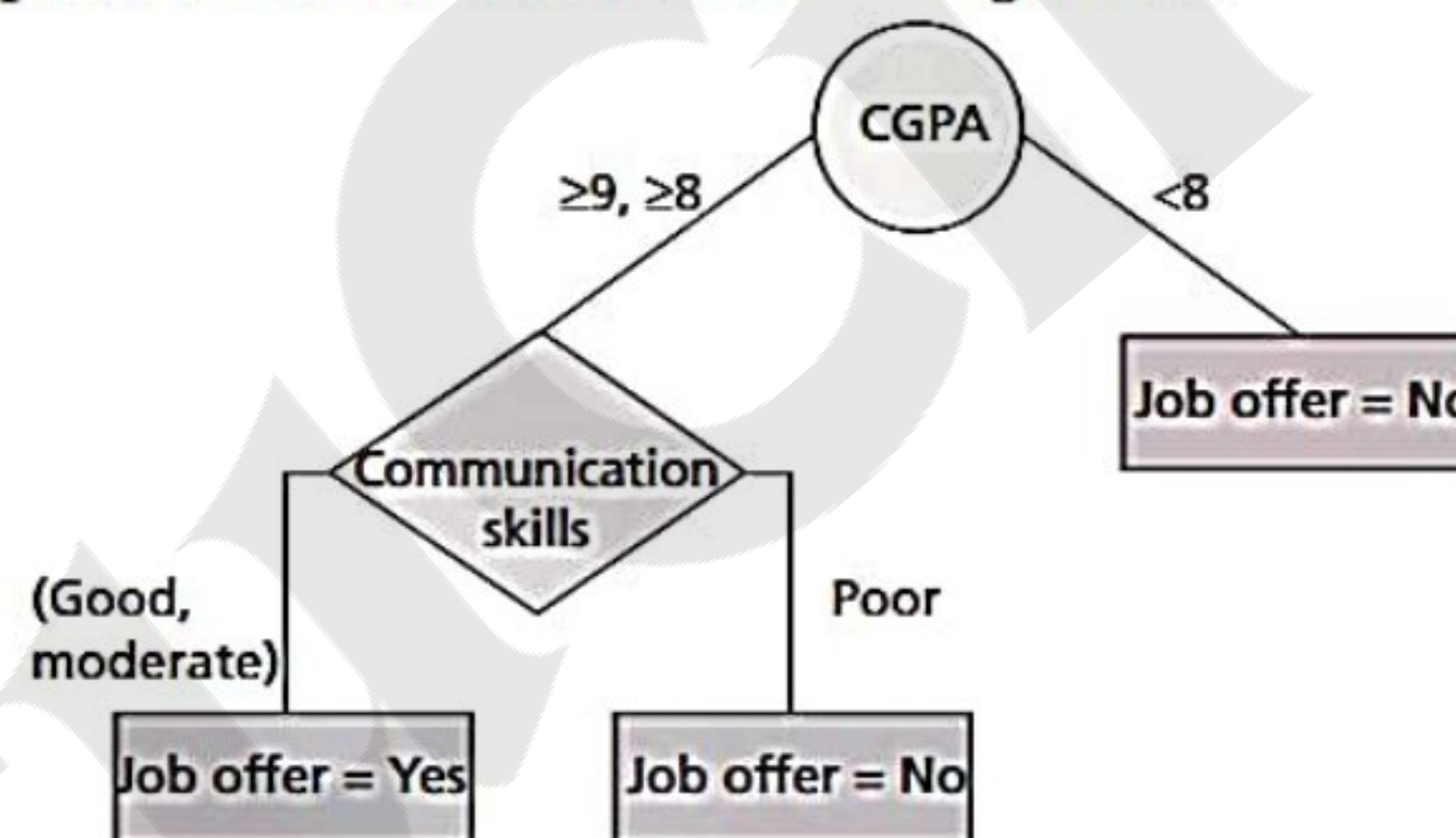


Figure 6.8: Final Tree

## 6.2.4 Regression Trees

Regression trees are a variant of decision trees where the target feature is a continuous valued variable. These trees can be constructed using an algorithm called reduction in variance which uses standard deviation to choose the best splitting attribute.

### Algorithm 6.5: Procedure for Constructing Regression Trees

1. Compute standard deviation for each attribute with respect to target attribute.
2. Compute standard deviation for the number of data instances of each distinct value of an attribute.
3. Compute weighted standard deviation for each attribute.

(Continued)

4. Compute standard deviation reduction by subtracting weighted standard deviation for each attribute from standard deviation of each attribute.
5. Choose the attribute with a higher standard deviation reduction as the best split attribute.
6. The best split attribute is placed as the root node.
7. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into different subsets.
8. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

**Example 6.6:** Construct a regression tree using the following Table 6.30 which consists of 10 data instances and 3 attributes 'Assessment', 'Assignment' and 'Project'. The target attribute is the 'Result' which is a continuous attribute.

Table 6.30: Training Dataset

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 1.    | Good       | Yes        | Yes     | 95         |
| 2.    | Average    | Yes        | No      | 70         |
| 3.    | Good       | No         | Yes     | 75         |
| 4.    | Poor       | No         | No      | 45         |
| 5.    | Good       | Yes        | Yes     | 98         |
| 6.    | Average    | No         | Yes     | 80         |
| 7.    | Good       | No         | No      | 75         |
| 8.    | Poor       | Yes        | Yes     | 65         |
| 9.    | Average    | No         | No      | 58         |
| 10.   | Good       | Yes        | Yes     | 89         |

**Solution:**

**Step 1:** Compute standard deviation for each attribute with respect to the target attribute:

$$\text{Average} = (95 + 70 + 75 + 45 + 98 + 80 + 75 + 65 + 58 + 89) = 75$$

$$\text{Standard Deviation} = \sqrt{\frac{(95 - 75)^2 + (70 - 75)^2 + (75 - 75)^2 + (45 - 75)^2 + (98 - 75)^2 + (80 - 75)^2 + (75 - 75)^2 + (65 - 75)^2 + (58 - 75)^2 + (89 - 75)^2}{10}}$$

$$= 16.55$$

**Assessment = Good (Table 6.31)**

Table 6.31: Attribute Assessment = Good

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 1.    | Good       | Yes        | Yes     | 95         |
| 3.    | Good       | No         | Yes     | 75         |
| 5.    | Good       | Yes        | Yes     | 98         |
| 7.    | Good       | No         | No      | 75         |
| 10.   | Good       | Yes        | Yes     | 89         |

$$\text{Average} = (95 + 75 + 98 + 75 + 89) = 86.4$$

$$\text{Standard Deviation} = \sqrt{\frac{(95 - 86.4)^2 + (75 - 86.4)^2 + (98 - 86.4)^2 + (75 - 86.4)^2 + (89 - 86.4)^2}{5}} = 10.9$$

**Assessment = Average (Table 6.32)**

**Table 6.32: Attribute Assessment = Average**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 2.    | Average    | Yes        | No      | 70         |
| 6.    | Average    | No         | Yes     | 80         |
| 9.    | Average    | No         | No      | 58         |

$$\text{Average} = (70 + 80 + 58) = 69.3$$

$$\text{Standard Deviation} = \sqrt{\frac{(70 - 69.3)^2 + (80 - 69.3)^2 + (58 - 69.3)^2}{3}} = 11.01$$

**Assessment = Poor (Table 6.33)**

**Table 6.33: Attribute Assessment = Poor**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 4.    | Poor       | No         | No      | 45         |
| 8.    | Poor       | Yes        | Yes     | 65         |

$$\text{Average} = (45 + 65) = 55$$

$$\text{Standard Deviation} = \sqrt{\frac{(45 - 55)^2 + (65 - 55)^2}{2}} = 14.14$$

Table 6.34 shows the standard deviation and data instances for the attribute-Assessment.

**Table 6.34: Standard Deviation for Assessment**

| Assessment | Standard Deviation | Data Instances |
|------------|--------------------|----------------|
| Good       | 10.9               | 5              |
| Average    | 11.01              | 3              |
| Poor       | 14.14              | 2              |

$$\text{Weighted standard deviation for Assessment} = \left(\frac{5}{10}\right) \times 10.9 + \left(\frac{3}{10}\right) \times 11.01 + \left(\frac{2}{10}\right) \times 14.14 = 11.58$$

$$\text{Standard deviation reduction for Assessment} = 16.55 - 11.58 = 4.97$$

**Assignment = Yes (Table 6.35)**

**Table 6.35: Assignment = Yes**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 1.    | Good       | Yes        | Yes     | 95         |
| 2.    | Average    | Yes        | No      | 70         |
| 5.    | Good       | Yes        | Yes     | 98         |
| 8.    | Poor       | Yes        | Yes     | 65         |
| 10.   | Good       | Yes        | Yes     | 89         |

$$\text{Average} = (95 + 70 + 98 + 65 + 89) = 83.4$$

$$\text{Standard Deviation} = \sqrt{\frac{(95 - 83.4)^2 + (70 - 83.4)^2 + (98 - 83.4)^2 + (65 - 83.4)^2 + (89 - 83.4)^2}{5}} \\ = 14.98$$

**Assignment = No (Table 6.36)**

**Table 6.36: Assignment = No**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 3.    | Good       | No         | Yes     | 75         |
| 4.    | Poor       | No         | No      | 45         |
| 6.    | Average    | No         | Yes     | 80         |
| 7.    | Good       | No         | No      | 75         |
| 9.    | Average    | No         | No      | 58         |

$$\text{Average} = (75 + 45 + 80 + 75 + 58) = 66.6$$

$$\text{Standard Deviation} = \sqrt{\frac{(75 - 66.6)^2 + (45 - 66.6)^2 + (80 - 66.6)^2 + (75 - 66.6)^2 + (58 - 66.6)^2}{5}} \\ = 14.7$$

Table 6.37 shows the Standard Deviation and Data Instances for attribute, Assignment.

**Table 6.37: Standard Deviation for Assignment**

| Assessment | Standard Deviation | Data Instances |
|------------|--------------------|----------------|
| Yes        | 14.98              | 5              |
| No         | 14.7               | 5              |

$$\text{Weighted standard deviation for Assignment} = \left(\frac{5}{10}\right) \times 14.98 + \left(\frac{5}{10}\right) \times 14.7 = 14.84$$

$$\text{Standard deviation reduction for Assignment} = 16.55 - 14.84 = 1.71$$

**Project = Yes (Table 6.38)**

**Table 6.38: Project = Yes**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 1.    | Good       | Yes        | Yes     | 95         |
| 3.    | Good       | No         | Yes     | 75         |
| 5.    | Good       | Yes        | Yes     | 98         |
| 6.    | Average    | No         | Yes     | 80         |
| 8.    | Poor       | Yes        | Yes     | 65         |
| 10.   | Good       | Yes        | Yes     | 89         |

$$\text{Average} = (95 + 75 + 98 + 80 + 65 + 89) = 83.7$$

$$\text{Standard Deviation} = \sqrt{\frac{(95 - 83.7)^2 + (75 - 83.7)^2 + (98 - 83.7)^2 + (80 - 83.7)^2 + (65 - 83.7)^2 + (89 - 83.7)^2}{6}} = 12.6$$

**Project = No (Table 6.39)**

**Table 6.39: Project = No**

| S.No. | Assessment | Assignment | Project | Result (%) |
|-------|------------|------------|---------|------------|
| 2.    | Average    | Yes        | No      | 70         |
| 4.    | Poor       | No         | No      | 45         |
| 7.    | Good       | No         | No      | 75         |
| 9.    | Average    | No         | No      | 58         |

$$\text{Average} = (70 + 45 + 75 + 58) = 62$$

$$\text{Standard Deviation} = \sqrt{\frac{(70 - 62)^2 + (45 - 62)^2 + (75 - 62)^2 + (58 - 62)^2}{4}} = 13.39$$

Table 6.40 shows the Standard Deviation and Data Instances for attribute, Project.

**Table 6.40: Standard Deviation for Project**

| Project | Standard Deviation | Data Instances |
|---------|--------------------|----------------|
| Yes     | 12.6               | 6              |
| No      | 13.39              | 4              |

$$\text{Weighted standard deviation for Assessment} = \left(\frac{6}{10}\right) \times 2.6 + \left(\frac{4}{10}\right) \times 13.39 = 12.92$$

$$\text{Standard deviation reduction for Assessment} = 16.55 - 12.92 = 3.63$$

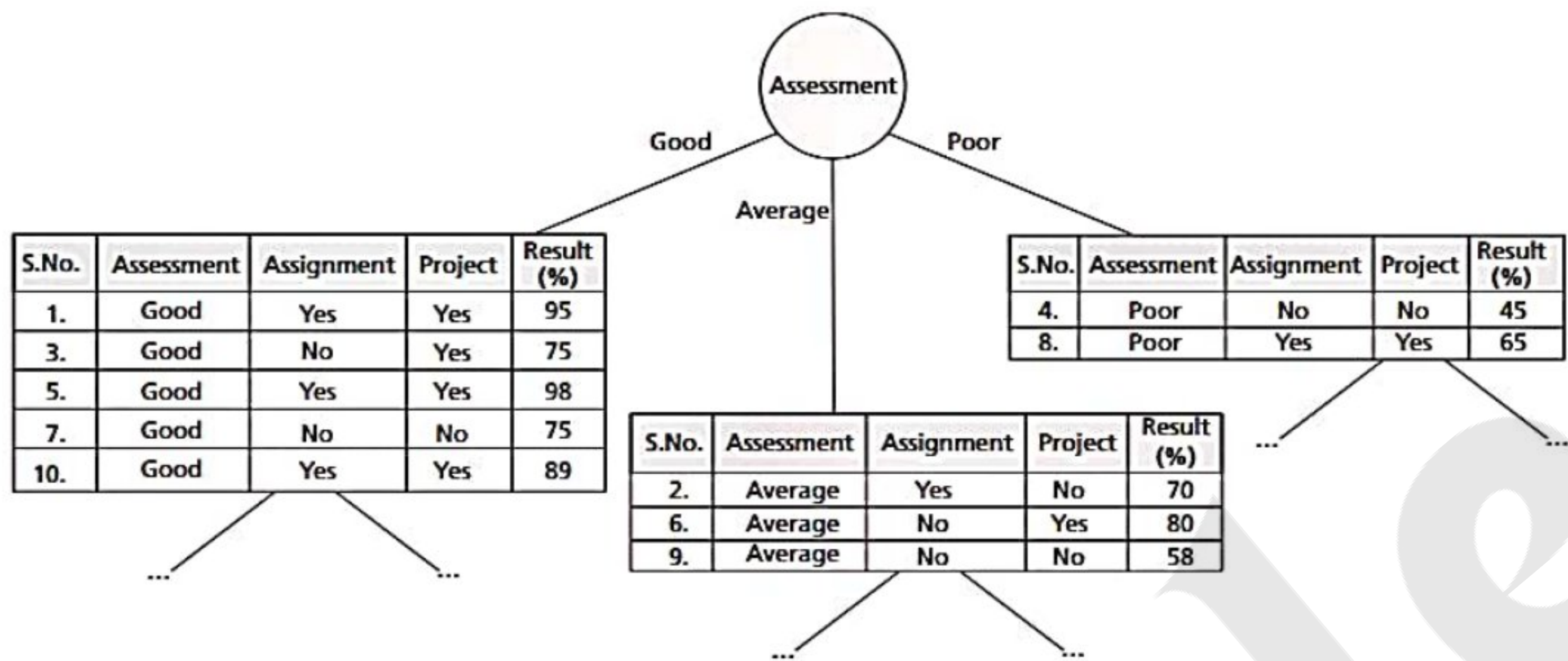
Table 6.41 shows the standard deviation reduction for each attribute in the training dataset.

**Table 6.41: Standard Deviation Reduction for Each Attribute**

| Attributes | Standard Deviation Reduction |
|------------|------------------------------|
| Assessment | 4.97                         |
| Assignment | 1.71                         |
| Project    | 3.63                         |

The attribute 'Assessment' has the maximum Standard Deviation Reduction and hence it is chosen as the best splitting attribute.

The training dataset is split into subsets based on the attribute 'Assessment' and this process is continued until the entire tree is constructed. Figure 6.9 shows the regression tree with 'Assessment' as the root node and the subsets in each branch.



**Figure 6.9:** Regression Tree with Assessment as Root Node

The rest of regression tree construction can be done as an exercise.